

AHDS - POWERSHELL FOR HARDWARE & EMBEDDED ENGINEERING

660 Interview Questions & Answers - FR / EN

Windows PowerShell 5.1 • PowerShell 7 • ISE • VS Code • Hardware automation • Production testing

Format: Question FR + Réponse FR | Same Question EN + Answer EN
Code examples are shown only once when the source code is language-independent.

Sommaire / Contents

- 01. Écosystème et versions PowerShell / PowerShell ecosystem and versions — Q1–Q10**
- 02. Hôtes, console, ISE et VS Code / Hosts, console, ISE, and VS Code — Q11–Q20
- 03. Aide et découverte / Help and discovery — Q21–Q30
- 04. Cmdlets, syntaxe et paramètres / Cmdlets, syntax, and parameters — Q31–Q40
- 05. Objets, propriétés et méthodes / Objects, properties, and methods — Q41–Q50
- 06. Pipeline et traitement objet / Pipeline and object processing — Q51–Q60
- 07. Variables, types et portée / Variables, types, and scope — Q61–Q70
- 08. Chaînes, guillemets et interpolation / Strings, quoting, and interpolation — Q71–Q80
- 09. Tableaux, collections, hashtables et objets personnalisés / Arrays, collections, hashtables, and custom objects — Q81–Q90
- 10. Opérateurs et conditions / Operators and conditions — Q91–Q100
- 11. Boucles et itération / Loops and iteration — Q101–Q110
- 12. Fonctions et paramètres / Functions and parameters — Q111–Q120
- 13. Fonctions avancées et binding / Advanced functions and binding — Q121–Q130
- 14. Scripts, arguments et contexte d'exécution / Scripts, arguments, and execution context — Q131–Q140
- 15. Modules et réutilisation / Modules and reuse — Q141–Q150**
- 16. Profils et préférences / Profiles and preferences — Q151–Q160
- 17. Gestion des erreurs / Error handling — Q161–Q170
- 18. Streams, output et redirection / Streams, output, and redirection — Q171–Q180
- 19. Fichiers, chemins et providers / Files, paths, and providers — Q181–Q190
- 20. CSV, JSON et XML / CSV, JSON, and XML — Q191–Q200
- 21. Texte et expressions régulières / Text and regular expressions — Q201–Q210
- 22. Processus, services et tâches planifiées / Processes, services, and scheduled tasks — Q211–Q220
- 23. Registre et environnement Windows / Windows Registry and environment — Q221–Q230
- 24. Réseau et diagnostics TCP/IP / Networking and TCP/IP diagnostics — Q231–Q240
- 25. Web, HTTP et API REST / Web, HTTP, and REST APIs — Q241–Q250
- 26. WMI et CIM / WMI and CIM — Q251–Q260
- 27. PnP, USB et Hardware IDs / PnP, USB, and hardware IDs — Q261–Q270
- 28. Ports série et communication COM / Serial ports and COM communication — Q271–Q280
- 29. Journaux Windows et diagnostic système / Windows logs and system diagnostics — Q281–Q290
- 30. Remoting, jobs et parallélisme / Remoting, jobs, and parallelism — Q291–Q300**
- 31. Sécurité, execution policy et credentials / Security, execution policy, and credentials — Q301–Q310**
- 32. PowerShell ISE, VS Code et debugging / PowerShell ISE, VS Code, and debugging — Q311–Q320
- 33. Git, build et toolchains / Git, build, and toolchains — Q321–Q330
- 34. Automatisation hardware et instruments / Hardware and instrument automation — Q331–Q340
- 35. Firmware flashing et programmation / Firmware flashing and programming — Q341–Q350
- 36. Stations de test et traçabilité production / Production test stations and traceability — Q351–Q360
- 37. Analyse de code - pipeline et objets / Code analysis - pipeline and objects — Q361–Q380
- 38. Analyse de code - collections et résultats structurés / Code analysis - collections and structured results — Q381–Q400
- 39. Analyse de code - fichiers et chemins / Code analysis - files and paths — Q401–Q420
- 40. Analyse de code - texte, regex et parsing / Code analysis - text, regex, and parsing — Q421–Q440
- 41. Analyse de code - erreurs et nettoyage / Code analysis - errors and cleanup — Q441–Q460
- 42. Analyse de code - programmes natifs et exit codes / Code analysis - native programs and exit codes — Q461–Q480
- 43. Analyse de code - registre et configuration Windows / Code analysis - registry and Windows configuration — Q481–Q500**
- 44. Analyse de code - réseau et API / Code analysis - networking and APIs — Q501–Q520
- 45. Analyse de code - PnP et USB / Code analysis - PnP and USB — Q521–Q540
- 46. Analyse de code - protocoles série / Code analysis - serial protocols — Q541–Q560
- 47. Analyse de code - fonctions de test / Code analysis - test functions — Q561–Q580
- 48. Analyse de code - jobs et parallélisme / Code analysis - jobs and parallelism — Q581–Q600
- 49. Analyse de code - programmation firmware / Code analysis - firmware programming — Q601–Q620
- 50. Analyse de code - production et traçabilité / Code analysis - production and traceability — Q621–Q640
- 51. Débogage - trouvez et corrigez / Debugging - find and fix — Q641–Q660

01. Écosystème et versions PowerShell / PowerShell ecosystem and versions

Q1 — FR

Que signifie / à quoi sert « Windows PowerShell 5.1 » en PowerShell ?

Réponse: Version historique intégrée à Windows, basée sur .NET Framework et encore importante pour les modules Windows hérités. Exemple: `$PSVersionTable.PSVersion`.

Q1 — EN

What does “Windows PowerShell 5.1” mean or do in PowerShell?

Answer: Legacy Windows-integrated edition based on .NET Framework and still important for older Windows modules. Example: `$PSVersionTable.PSVersion`.

Q2 — FR

Que signifie / à quoi sert « PowerShell 7 » en PowerShell ?

Réponse: Édition moderne, multiplateforme et basée sur .NET moderne; elle s'installe séparément de Windows PowerShell 5.1. Exemple: `pwsh`.

Q2 — EN

What does “PowerShell 7” mean or do in PowerShell?

Answer: Modern cross-platform edition based on modern .NET; it installs side-by-side with Windows PowerShell 5.1. Example: `pwsh`.

Q3 — FR

Que signifie / à quoi sert « powershell.exe » en PowerShell ?

Réponse: Exécutable de Windows PowerShell 5.1 sur Windows. Exemple: `powershell.exe -NoProfile`.

Q3 — EN

What does “powershell.exe” mean or do in PowerShell?

Answer: Executable for Windows PowerShell 5.1 on Windows. Example: `powershell.exe -NoProfile`.

Q4 — FR

Que signifie / à quoi sert « pwsh.exe » en PowerShell ?

Réponse: Exécutable de PowerShell 7 sur Windows. Exemple: `pwsh -NoProfile`.

Q4 — EN

What does “pwsh.exe” mean or do in PowerShell?

Answer: Executable for PowerShell 7 on Windows. Example: `pwsh -NoProfile`.

Q5 — FR

Que signifie / à quoi sert « \$PSVersionTable » en PowerShell ?

Réponse: Variable automatique qui décrit la version PowerShell, l'édition et des informations de plateforme. Exemple: `$PSVersionTable`.

Q5 — EN

What does “\$PSVersionTable” mean or do in PowerShell?

Answer: Automatic variable that reports PowerShell version, edition, and platform information. Example: `$PSVersionTable`.

Q6 — FR

Que signifie / à quoi sert « PSEdition » en PowerShell ?

Réponse: Indique notamment Desktop pour Windows PowerShell et Core pour PowerShell 7. Exemple: `$PSVersionTable.PSEdition`.

Q6 — EN

What does “PSEdition” mean or do in PowerShell?

Answer: Indicates Desktop for Windows PowerShell and Core for PowerShell 7. Example: `$PSVersionTable.PSEdition`.

Q7 — FR

Que signifie / à quoi sert « compatibilité des modules » en PowerShell ?

Réponse: Un module prévu pour .NET Framework/Windows PowerShell peut ne pas fonctionner nativement sous PowerShell 7; il faut tester la compatibilité.

Q7 — EN

What does “compatibilité des modules” mean or do in PowerShell?

Answer: A module written for .NET Framework/Windows PowerShell may not run natively in PowerShell 7; compatibility must be tested.

Q8 — FR

Que signifie / à quoi sert « side-by-side » en PowerShell ?

Réponse: Windows PowerShell 5.1 et PowerShell 7 peuvent coexister sur la même machine, ce qui est utile pour conserver les anciens outils tout en automatisant avec la version moderne.

Q8 — EN

What does “side-by-side” mean or do in PowerShell?

Answer: Windows PowerShell 5.1 and PowerShell 7 can coexist on the same machine, preserving legacy tooling while enabling modern automation.

Q9 — FR

Que signifie / à quoi sert « cross-platform » en PowerShell ?

Réponse: PowerShell 7 fonctionne sur Windows, Linux et macOS, mais les cmdlets qui gèrent des composants Windows restent naturellement spécifiques à Windows.

Q9 — EN

What does “cross-platform” mean or do in PowerShell?

Answer: PowerShell 7 runs on Windows, Linux, and macOS, while cmdlets that manage Windows components remain Windows-specific.

Q10 — FR

Que signifie / à quoi sert « LTS » en PowerShell ?

Q10 — EN

What does “LTS” mean or do in PowerShell?

Réponse: Certaines versions de PowerShell bénéficient d'un support long terme; pour une station de production, on privilégie souvent une version supportée et validée plutôt que la toute dernière version non testée.

Answer: Some PowerShell releases receive long-term support; production stations often favor a supported validated version over an untested newest release.

02. Hôtes, console, ISE et VS Code / Hosts, console, ISE, and VS Code

Q11 — FR

Que signifie / à quoi sert « ConsoleHost » en PowerShell ?

Réponse: Hôte console classique dans lequel PowerShell peut s'exécuter.

Q11 — EN

What does “ConsoleHost” mean or do in PowerShell?

Answer: Classic console host in which PowerShell can run.

Q12 — FR

Que signifie / à quoi sert « Windows Terminal » en PowerShell ?

Réponse: Terminal moderne pouvant héberger PowerShell 7, Windows PowerShell, CMD et WSL dans des onglets séparés.

Q12 — EN

What does “Windows Terminal” mean or do in PowerShell?

Answer: Modern terminal that can host PowerShell 7, Windows PowerShell, CMD, and WSL in separate tabs.

Q13 — FR

Que signifie / à quoi sert « PowerShell ISE » en PowerShell ?

Réponse: Environnement graphique historique pour Windows PowerShell avec éditeur, console, breakpoints et debugging. Ce n'est pas une autre variante du langage PowerShell.

Q13 — EN

What does “PowerShell ISE” mean or do in PowerShell?

Answer: Legacy graphical environment for Windows PowerShell with editor, console, breakpoints, and debugging. It is not a separate PowerShell language variant.

Q14 — FR

Que signifie / à quoi sert « limite de PowerShell ISE » en PowerShell ?

Réponse: ISE est lié à Windows PowerShell et n'est pas l'environnement recommandé pour PowerShell 7; VS Code avec l'extension PowerShell est le choix moderne.

Q14 — EN

What does “limite de PowerShell ISE” mean or do in PowerShell?

Answer: ISE is tied to Windows PowerShell and is not the recommended environment for PowerShell 7; VS Code with the PowerShell extension is the modern choice.

Q15 — FR

Que signifie / à quoi sert « VS Code + PowerShell extension » en PowerShell ?

Réponse: Fournit coloration, IntelliSense, terminal intégré, debugging, analyse de scripts et support de PowerShell 7.

Q15 — EN

What does “VS Code + PowerShell extension” mean or do in PowerShell?

Answer: Provides syntax highlighting, IntelliSense, integrated terminal, debugging, script analysis, and PowerShell 7 support.

Q16 — FR

Que signifie / à quoi sert « Integrated Console » en PowerShell ?

Réponse: Dans VS Code, la PowerShell Integrated Console est pilotée par l'extension et sert au debugging et à l'exécution interactive.

Q16 — EN

What does “Integrated Console” mean or do in PowerShell?

Answer: In VS Code, the PowerShell Integrated Console is managed by the extension for debugging and interactive execution.

Q17 — FR

Que signifie / à quoi sert « profil par hôte » en PowerShell ?

Réponse: Les profils PowerShell peuvent être différents selon l'utilisateur et l'hôte; un script de production ne doit pas dépendre silencieusement d'un profil personnel.

Q17 — EN

What does “profil par hôte” mean or do in PowerShell?

Answer: PowerShell profiles can differ by user and host; a production script should not silently depend on a personal profile.

Q18 — FR

Que signifie / à quoi sert « -NoProfile » en PowerShell ?

Réponse: Démarre PowerShell sans charger les profils, ce qui améliore la reproductibilité des scripts de test. Exemple: pwsh -NoProfile -File test.ps1.

Q18 — EN

What does “-NoProfile” mean or do in PowerShell?

Answer: Starts PowerShell without loading profiles, improving reproducibility for test scripts. Example: pwsh -NoProfile -File test.ps1.

Q19 — FR

Que signifie / à quoi sert « STA/MTA » en PowerShell ?

Réponse: Certains composants .NET, COM ou UI peuvent dépendre du modèle d'appartement; c'est surtout pertinent pour des outils GUI/COM spécialisés.

Q19 — EN

What does “STA/MTA” mean or do in PowerShell?

Answer: Some .NET, COM, or UI components can depend on apartment state; this mainly matters for specialized GUI/COM tools.

Q20 — FR

Que signifie / à quoi sert « terminal vs shell » en PowerShell ?

Réponse: Le terminal affiche et transporte l'entrée/sortie; PowerShell est le shell et langage qui interprète les commandes.

Q20 — EN

What does “terminal vs shell” mean or do in PowerShell?

Answer: The terminal displays and transports input/output; PowerShell is the shell and language that interprets commands.

03. Aide et découverte / Help and discovery

Q21 — FR

Que signifie / à quoi sert « Get-Help » en PowerShell ?

Réponse: Affiche l'aide d'une commande, avec syntaxe, paramètres, exemples et notes. Exemple: Get-Help Get-Process -Examples.

Q21 — EN

What does “Get-Help” mean or do in PowerShell?

Answer: Displays command help including syntax, parameters, examples, and notes. Example: Get-Help Get-Process -Examples.

Q22 — FR

Que signifie / à quoi sert « Update-Help » en PowerShell ?

Réponse: Télécharge ou met à jour les fichiers d'aide disponibles pour les modules installés. Exemple: Update-Help.

Q22 — EN

What does “Update-Help” mean or do in PowerShell?

Answer: Downloads or updates available help files for installed modules. Example: Update-Help.

Q23 — FR

Que signifie / à quoi sert « Get-Command » en PowerShell ?

Réponse: Découvre cmdlets, fonctions, alias, scripts et exécutables visibles dans la session. Exemple: Get-Command *Serial*.

Q23 — EN

What does “Get-Command” mean or do in PowerShell?

Answer: Discovers cmdlets, functions, aliases, scripts, and executables visible in the session. Example: Get-Command *Serial*.

Q24 — FR

Que signifie / à quoi sert « Get-Member » en PowerShell ?

Réponse: Affiche le type, les propriétés et les méthodes des objets passant dans le pipeline. Exemple: Get-Process | Get-Member.

Q24 — EN

What does “Get-Member” mean or do in PowerShell?

Answer: Shows the type, properties, and methods of objects in the pipeline. Example: Get-Process | Get-Member.

Q25 — FR

Que signifie / à quoi sert « Get-Alias » en PowerShell ?

Réponse: Affiche les alias et leur commande cible. Exemple: Get-Alias ls.

Q25 — EN

What does “Get-Alias” mean or do in PowerShell?

Answer: Displays aliases and the commands they map to. Example: Get-Alias ls.

Q26 — FR

Que signifie / à quoi sert « Get-Module » en PowerShell ?

Réponse: Liste les modules chargés ou disponibles. Exemple: Get-Module -ListAvailable.

Q26 — EN

What does “Get-Module” mean or do in PowerShell?

Answer: Lists loaded or available modules. Example: Get-Module -ListAvailable.

Q27 — FR

Que signifie / à quoi sert « Find-Module » en PowerShell ?

Réponse: Recherche des modules dans un dépôt enregistré tel que PowerShell Gallery. Exemple: Find-Module Pester.

Q27 — EN

What does “Find-Module” mean or do in PowerShell?

Answer: Searches registered repositories such as PowerShell Gallery for modules. Example: Find-Module Pester.

Q28 — FR

Que signifie / à quoi sert « Get-PSDrive » en PowerShell ?

Réponse: Affiche les lecteurs PowerShell, y compris FileSystem, Registry et autres providers. Exemple: Get-PSDrive.

Q28 — EN

What does “Get-PSDrive” mean or do in PowerShell?

Answer: Lists PowerShell drives including FileSystem, Registry, and other providers. Example: Get-PSDrive.

Q29 — FR

Que signifie / à quoi sert « Get-PSProvider » en PowerShell ?

Réponse: Liste les providers qui exposent différentes sources de données comme des lecteurs. Exemple: Get-PSProvider.

Q29 — EN

What does “Get-PSProvider” mean or do in PowerShell?

Answer: Lists providers that expose different data stores as drive-like namespaces. Example: Get-PSProvider.

Q30 — FR

Que signifie / à quoi sert « Tab completion » en PowerShell ?

Réponse: La complétion réduit les erreurs de frappe et aide à découvrir paramètres et valeurs, mais elle ne remplace pas la lecture de l'aide.

Q30 — EN

What does “Tab completion” mean or do in PowerShell?

Answer: Tab completion reduces typing errors and helps discover parameters and values, but it does not replace reading help.

04. Cmdlets, syntaxe et paramètres / Cmdlets, syntax, and parameters

Q31 — FR

Que signifie / à quoi sert « Verb-Noun » en PowerShell ?

Réponse: Convention de nommage des cmdlets, par exemple Get-Process ou Set-Item.

Q31 — EN

What does “Verb-Noun” mean or do in PowerShell?

Answer: Cmdlet naming convention such as Get-Process or Set-Item.

Q32 — FR

Que signifie / à quoi sert « Get-Verb » en PowerShell ?

Réponse: Affiche les verbes PowerShell approuvés et leur groupe sémantique. Exemple: Get-Verb.

Q32 — EN

What does “Get-Verb” mean or do in PowerShell?

Answer: Lists approved PowerShell verbs and their semantic groups. Example: Get-Verb.

Q33 — FR

Que signifie / à quoi sert « paramètre nommé » en PowerShell ?

Réponse: Paramètre explicitement introduit par son nom, plus lisible dans les scripts de production. Exemple: Get-ChildItem -Path C:\Logs -Filter *.csv.

Q33 — EN

What does “paramètre nommé” mean or do in PowerShell?

Answer: Parameter explicitly introduced by name, improving readability in production scripts. Example: Get-ChildItem -Path C:\Logs -Filter *.csv.

Q34 — FR

Que signifie / à quoi sert « paramètre positionnel » en PowerShell ?

Réponse: Paramètre fourni sans son nom; pratique en interactif mais moins explicite dans un script maintenable.

Q34 — EN

What does “paramètre positionnel” mean or do in PowerShell?

Answer: Parameter supplied without its name; convenient interactively but less explicit in maintainable scripts.

Q35 — FR

Que signifie / à quoi sert « switch parameter » en PowerShell ?

Réponse: Paramètre booléen présent/absent, par exemple -Force ou -Recurse.

Q35 — EN

What does “switch parameter” mean or do in PowerShell?

Answer: Boolean present/absent parameter such as -Force or -Recurse.

Q36 — FR

Que signifie / à quoi sert « CommonParameters » en PowerShell ?

Réponse: Paramètres communs comme -Verbose, -Debug, -ErrorAction et -WarningAction disponibles sur les fonctions avancées/cmdlets.

Q36 — EN

What does “CommonParameters” mean or do in PowerShell?

Answer: Common parameters such as -Verbose, -Debug, -ErrorAction, and -WarningAction available on cmdlets/advanced functions.

Q37 — FR

Que signifie / à quoi sert « -WhatIf » en PowerShell ?

Réponse: Simule de nombreuses opérations modifiant l'état sans les exécuter réellement lorsque la commande prend en charge ShouldProcess.

Q37 — EN

What does “-WhatIf” mean or do in PowerShell?

Answer: Simulates many state-changing operations without actually performing them when the command supports ShouldProcess.

Q38 — FR

Que signifie / à quoi sert « -Confirm » en PowerShell ?

Q38 — EN

What does “-Confirm” mean or do in PowerShell?

Réponse: Demande une confirmation avant une opération prenant en charge ShouldProcess.

Answer: Requests confirmation before an operation that supports ShouldProcess.

Q39 — FR

Que signifie / à quoi sert « splatting » en PowerShell ?

Réponse: Passe un ensemble de paramètres via un hashtable ou un tableau, utile pour rendre les appels longs plus lisibles.

Q39 — EN

What does “splatting” mean or do in PowerShell?

Answer: Passes parameter sets using a hashtable or array, making long calls easier to read.

Q40 — FR

Que signifie / à quoi sert « parameter set » en PowerShell ?

Réponse: Certaines cmdlets ont plusieurs combinaisons exclusives de paramètres; l'aide indique les parameter sets valides.

Q40 — EN

What does “parameter set” mean or do in PowerShell?

Answer: Some cmdlets have multiple mutually exclusive parameter combinations; help shows valid parameter sets.

05. Objets, propriétés et méthodes / Objects, properties, and methods

Q41 — FR

Que signifie / à quoi sert « objet PowerShell » en PowerShell ?

Réponse: PowerShell transmet généralement des objets .NET structurés plutôt que de simples lignes de texte.

Q41 — EN

What does “objet PowerShell” mean or do in PowerShell?

Answer: PowerShell usually passes structured .NET objects rather than plain text lines.

Q42 — FR

Que signifie / à quoi sert « property » en PowerShell ?

Réponse: Valeur descriptive d'un objet, par exemple Process.Id ou Service.Status.

Q42 — EN

What does “property” mean or do in PowerShell?

Answer: Descriptive value on an object, such as Process.Id or Service.Status.

Q43 — FR

Que signifie / à quoi sert « method » en PowerShell ?

Réponse: Action exposée par un objet .NET, par exemple une méthode Close() ou ToString().

Q43 — EN

What does “method” mean or do in PowerShell?

Answer: Action exposed by a .NET object, such as Close() or ToString().

Q44 — FR

Que signifie / à quoi sert « PSOBJECT » en PowerShell ?

Réponse: Couche d'adaptation PowerShell qui permet d'enrichir des objets avec des propriétés ou méthodes étendues.

Q44 — EN

What does “PSObject” mean or do in PowerShell?

Answer: PowerShell adaptation layer that can enrich objects with extended properties or methods.

Q45 — FR

Que signifie / à quoi sert « Select-Object » en PowerShell ?

Réponse: Sélectionne propriétés, crée des propriétés calculées ou limite un ensemble d'objets. Exemple: Get-Process | Select-Object Name,Id.

Q45 — EN

What does “Select-Object” mean or do in PowerShell?

Answer: Selects properties, creates calculated properties, or limits an object set. Example: Get-Process | Select-Object Name,Id.

Q46 — FR

Que signifie / à quoi sert « Format-Table » en PowerShell ?

Réponse: Formate des objets pour affichage tabulaire; il faut généralement le garder à la fin d'un pipeline destiné à l'écran.

Q46 — EN

What does “Format-Table” mean or do in PowerShell?

Answer: Formats objects as a table; it should generally stay at the end of a display-oriented pipeline.

Q47 — FR

Que signifie / à quoi sert « Format-List » en PowerShell ?

Réponse: Formate des objets en liste verticale pour inspection détaillée.

Q47 — EN

What does “Format-List” mean or do in PowerShell?

Answer: Formats objects as a vertical list for detailed inspection.

Q48 — FR

Que signifie / à quoi sert « calculated property » en PowerShell ?

Q48 — EN

What does “calculated property” mean or do in PowerShell?

Réponse: Propriété créée avec un hashtable Name/Expression dans Select-Object ou Format-Table.

Answer: Property created with a Name/Expression hashtable in Select-Object or Format-Table.

Q49 — FR

Que signifie / à quoi sert « type accelerator » en PowerShell ?

Réponse: Alias de type .NET comme [int], [string], [datetime], [xml] ou [regex].

Q49 — EN

What does “type accelerator” mean or do in PowerShell?

Answer: Short .NET type names such as [int], [string], [datetime], [xml], or [regex].

Q50 — FR

Que signifie / à quoi sert « GetType() » en PowerShell ?

Réponse: Méthode .NET qui permet d'identifier le type réel d'un objet pendant le debug. Exemple: \$x.GetType().FullName.

Q50 — EN

What does “GetType()” mean or do in PowerShell?

Answer: .NET method used to identify an object's actual type during debugging. Example: \$x.GetType().FullName.

06. Pipeline et traitement objet / Pipeline and object processing

Q51 — FR

Que signifie / à quoi sert « | » en PowerShell ?

Réponse: Envoie les objets produits à gauche vers la commande à droite.

Q51 — EN

What does “|” mean or do in PowerShell?

Answer: Sends objects produced on the left to the command on the right.

Q52 — FR

Que signifie / à quoi sert « Where-Object » en PowerShell ?

Réponse: Filtre des objets selon une condition. Exemple: Get-Service | Where-Object Status -eq Running.

Q52 — EN

What does “Where-Object” mean or do in PowerShell?

Answer: Filters objects according to a condition. Example: Get-Service | Where-Object Status -eq Running.

Q53 — FR

Que signifie / à quoi sert « ForEach-Object » en PowerShell ?

Réponse: Exécute un bloc pour chaque objet du pipeline. Exemple: 1..5 | ForEach-Object { \$_ * 2 }.

Q53 — EN

What does “ForEach-Object” mean or do in PowerShell?

Answer: Runs a script block for each pipeline object. Example: 1..5 | ForEach-Object { \$_ * 2 }.

Q54 — FR

Que signifie / à quoi sert « \$_ / \$PSItem » en PowerShell ?

Réponse: Variable automatique représentant l'objet courant dans de nombreux script blocks du pipeline.

Q54 — EN

What does “\$_ / \$PSItem” mean or do in PowerShell?

Answer: Automatic variable representing the current pipeline object in many script blocks.

Q55 — FR

Que signifie / à quoi sert « Sort-Object » en PowerShell ?

Réponse: Trie les objets selon une ou plusieurs propriétés. Exemple: Get-Process | Sort-Object CPU -Descending.

Q55 — EN

What does “Sort-Object” mean or do in PowerShell?

Answer: Sorts objects by one or more properties. Example: Get-Process | Sort-Object CPU -Descending.

Q56 — FR

Que signifie / à quoi sert « Group-Object » en PowerShell ?

Réponse: Regroupe les objets selon une propriété et fournit un compteur par groupe.

Q56 — EN

What does “Group-Object” mean or do in PowerShell?

Answer: Groups objects by a property and returns a count per group.

Q57 — FR

Que signifie / à quoi sert « Measure-Object » en PowerShell ?

Réponse: Calcule count, sum, average, min/max sur des propriétés numériques ou compte texte/lignes.

Q57 — EN

What does “Measure-Object” mean or do in PowerShell?

Answer: Calculates count, sum, average, min/max on numeric properties or counts text/lines.

Q58 — FR

Que signifie / à quoi sert « Tee-Object » en PowerShell ?

Q58 — EN

What does “Tee-Object” mean or do in PowerShell?

Réponse: Copie les objets du pipeline vers un fichier ou une variable tout en continuant le pipeline.

Answer: Copies pipeline objects to a file or variable while allowing them to continue through the pipeline.

Q59 — FR

Que signifie / à quoi sert « pipeline binding » en PowerShell ?

Réponse: PowerShell peut lier l'objet entrant à un paramètre par valeur ou par nom de propriété.

Q59 — EN

What does “pipeline binding” mean or do in PowerShell?

Answer: PowerShell can bind incoming objects to parameters by value or by property name.

Q60 — FR

Que signifie / à quoi sert « éviter Format-* trop tôt » en PowerShell ?

Réponse: Après Format-Table/Format-List, le pipeline contient surtout des objets de formatage; exporter ou retraiter ensuite produit souvent un résultat incorrect.

Q60 — EN

What does “éviter Format-* trop tôt” mean or do in PowerShell?

Answer: After Format-Table/Format-List, the pipeline mainly contains formatting objects; exporting or further processing often produces incorrect results.

07. Variables, types et portée / Variables, types, and scope

Q61 — FR

Que signifie / à quoi sert « \$variable » en PowerShell ?

Réponse: Les variables commencent par \$ et peuvent contenir presque tout objet .NET. Exemple: \$count = 10.

Q61 — EN

What does “\$variable” mean or do in PowerShell?

Answer: Variables begin with \$ and can contain nearly any .NET object. Example: \$count = 10.

Q62 — FR

Que signifie / à quoi sert « [int] / [double] / [decimal] » en PowerShell ?

Réponse: Conversions ou contraintes de type utiles pour éviter des conversions implicites inattendues.

Q62 — EN

What does “[int] / [double] / [decimal]” mean or do in PowerShell?

Answer: Type casts or constraints useful for avoiding unexpected implicit conversions.

Q63 — FR

Que signifie / à quoi sert « [bool] » en PowerShell ?

Réponse: Type logique \$true/\$false; attention aux conversions de chaînes non vides qui deviennent souvent vraies.

Q63 — EN

What does “[bool]” mean or do in PowerShell?

Answer: Boolean type \$true/\$false; be careful because nonempty strings often convert to true.

Q64 — FR

Que signifie / à quoi sert « \$null » en PowerShell ?

Réponse: Représente l'absence de valeur; il est utile de tester \$null explicitement lors du parsing de données hardware.

Q64 — EN

What does “\$null” mean or do in PowerShell?

Answer: Represents no value; explicit \$null checks are useful when parsing hardware data.

Q65 — FR

Que signifie / à quoi sert « scope local » en PowerShell ?

Réponse: Portée courante d'une fonction/script block.

Q65 — EN

What does “scope local” mean or do in PowerShell?

Answer: Current function/script-block scope.

Q66 — FR

Que signifie / à quoi sert « scope script » en PowerShell ?

Réponse: Portée du fichier .ps1 ou module courant.

Q66 — EN

What does “scope script” mean or do in PowerShell?

Answer: Scope of the current .ps1 script or module.

Q67 — FR

Que signifie / à quoi sert « scope global » en PowerShell ?

Réponse: Portée de session globale; à éviter pour les scripts complexes car elle augmente les dépendances cachées.

Q67 — EN

What does “scope global” mean or do in PowerShell?

Answer: Global session scope; best avoided in complex scripts because it creates hidden dependencies.

Q68 — FR

Que signifie / à quoi sert « \$script:Name » en PowerShell ?

Q68 — EN

What does “\$script:Name” mean or do in PowerShell?

Réponse: Qualifie explicitement une variable dans la portée script.

Answer: Explicitly qualifies a variable in script scope.

Q69 — FR

Que signifie / à quoi sert « Set-StrictMode » en PowerShell ?

Réponse: Détecte certains usages fragiles comme variables non initialisées ou références de propriétés invalides selon la version choisie.

Q69 — EN

What does “Set-StrictMode” mean or do in PowerShell?

Answer: Detects some fragile behavior such as uninitialized variables or invalid property references depending on selected version.

Q70 — FR

Que signifie / à quoi sert « Remove-Variable » en PowerShell ?

Réponse: Supprime une variable de la portée indiquée.

Q70 — EN

What does “Remove-Variable” mean or do in PowerShell?

Answer: Removes a variable from the specified scope.

08. Chaînes, guillemets et interpolation / Strings, quoting, and interpolation

Q71 — FR

Que signifie / à quoi sert « simple quotes » en PowerShell ?

Réponse: Les chaînes entre apostrophes sont littérales: les variables ne sont pas interpolées. Exemple: '\$port'.

Q71 — EN

What does “simple quotes” mean or do in PowerShell?

Answer: Single-quoted strings are literal: variables are not expanded. Example: '\$port'.

Q72 — FR

Que signifie / à quoi sert « double quotes » en PowerShell ?

Réponse: Les chaînes entre guillemets doubles interpolent variables et subexpressions. Exemple: "Port=\$port".

Q72 — EN

What does “double quotes” mean or do in PowerShell?

Answer: Double-quoted strings expand variables and subexpressions. Example: "Port=\$port".

Q73 — FR

Que signifie / à quoi sert « \$() subexpression » en PowerShell ?

Réponse: Permet d'évaluer une expression complexe à l'intérieur d'une chaîne interpolée. Exemple: "Time=\$([datetime]::Now)".

Q73 — EN

What does “\$() subexpression” mean or do in PowerShell?

Answer: Evaluates a more complex expression inside an interpolated string. Example: "Time=\$([datetime]::Now)".

Q74 — FR

Que signifie / à quoi sert « backtick ` » en PowerShell ?

Réponse: Caractère d'échappement PowerShell pour certains caractères; il faut l'utiliser avec modération car il rend les scripts fragiles.

Q74 — EN

What does “backtick `” mean or do in PowerShell?

Answer: PowerShell escape character for some characters; use sparingly because it can make scripts fragile.

Q75 — FR

Que signifie / à quoi sert « here-string » en PowerShell ?

Réponse: Chaîne multilignes délimitée par @"..."@ ou @'...'@, pratique pour JSON, SQL, logs ou templates.

Q75 — EN

What does “here-string” mean or do in PowerShell?

Answer: Multiline string delimited by @"..."@ or @'...'@, useful for JSON, SQL, logs, or templates.

Q76 — FR

Que signifie / à quoi sert « -f format operator » en PowerShell ?

Réponse: Formate une chaîne avec des placeholders .NET et des spécificateurs numériques/date. Exemple: "0x{0:X4}" -f 4660.

Q76 — EN

What does “-f format operator” mean or do in PowerShell?

Answer: Formats strings using .NET placeholders and numeric/date format specifiers. Example: "0x{0:X4}" -f 4660.

Q77 — FR

Que signifie / à quoi sert « Join-Path » en PowerShell ?

Réponse: Construit des chemins sans concaténer manuellement les séparateurs. Exemple: Join-Path \$PSScriptRoot "logs".

Q77 — EN

What does “Join-Path” mean or do in PowerShell?

Answer: Builds paths without manually concatenating separators. Example: Join-Path \$PSScriptRoot "logs".

Q78 — FR

Que signifie / à quoi sert « \$PSScriptRoot » en PowerShell ?

Q78 — EN

What does “\$PSScriptRoot” mean or do in PowerShell?

Réponse: Répertoire contenant le script courant, utile pour rendre un package de test portable.

Answer: Directory containing the current script, useful for portable test packages.

Q79 — FR

Que signifie / à quoi sert « Split-Path » en PowerShell ?

Réponse: Extrait parent, leaf ou autres parties d'un chemin.

Q79 — EN

What does “Split-Path” mean or do in PowerShell?

Answer: Extracts parent, leaf, or other parts of a path.

Q80 — FR

Que signifie / à quoi sert « culture/locale » en PowerShell ?

Réponse: La conversion de nombres/dates peut dépendre de la culture; pour les logs machine-to-machine, utiliser des formats invariants explicites.

Q80 — EN

What does “culture/locale” mean or do in PowerShell?

Answer: Number/date conversion can depend on culture; machine-to-machine logs should use explicit invariant formats.

09. Tableaux, collections, hashtables et objets personnalisés / Arrays, collections, hashtables, and custom objects

Q81 — FR

Que signifie / à quoi sert « @() » en PowerShell ?

Réponse: Force ou crée un tableau PowerShell. Exemple: \$a = @(1,2,3).

Q81 — EN

What does “@()” mean or do in PowerShell?

Answer: Creates or forces a PowerShell array. Example: \$a = @(1,2,3).

Q82 — FR

Que signifie / à quoi sert « array indexing » en PowerShell ?

Réponse: Indexation à partir de zéro; les indices négatifs accèdent depuis la fin.

Q82 — EN

What does “array indexing” mean or do in PowerShell?

Answer: Zero-based indexing; negative indexes access from the end.

Q83 — FR

Que signifie / à quoi sert « range operator .. » en PowerShell ?

Réponse: Crée une séquence simple d'entiers ou caractères. Exemple: 1..10.

Q83 — EN

What does “range operator ..” mean or do in PowerShell?

Answer: Creates a simple integer or character sequence. Example: 1..10.

Q84 — FR

Que signifie / à quoi sert « ArrayList/List[T] » en PowerShell ?

Réponse: Collections .NET plus efficaces que la recréation répétée d'un tableau quand on ajoute de nombreux éléments.

Q84 — EN

What does “ArrayList/List[T]” mean or do in PowerShell?

Answer: .NET collections more efficient than repeatedly rebuilding an array when appending many items.

Q85 — FR

Que signifie / à quoi sert « @{} hashtable » en PowerShell ?

Réponse: Collection clé-valeur utilisée pour configuration, splatting et lookup.

Q85 — EN

What does “@{} hashtable” mean or do in PowerShell?

Answer: Key-value collection used for configuration, splatting, and lookup.

Q86 — FR

Que signifie / à quoi sert « [ordered]@{} » en PowerShell ?

Réponse: Hashtable ordonné qui préserve l'ordre d'insertion, utile pour rapports lisibles.

Q86 — EN

What does “[ordered]@{}” mean or do in PowerShell?

Answer: Ordered hashtable that preserves insertion order, useful for readable reports.

Q87 — FR

Que signifie / à quoi sert « [pscustomobject] » en PowerShell ?

Réponse: Crée rapidement un objet structuré avec propriétés nommées, idéal pour résultats de test.

Q87 — EN

What does “[pscustomobject]” mean or do in PowerShell?

Answer: Quickly creates a structured object with named properties, ideal for test results.

Q88 — FR

Que signifie / à quoi sert « Add-Member » en PowerShell ?

Q88 — EN

What does “Add-Member” mean or do in PowerShell?

Réponse: Ajoute dynamiquement propriétés ou méthodes étendues à un objet.

Answer: Dynamically adds extended properties or methods to an object.

Q89 — FR

Que signifie / à quoi sert « ContainsKey » en PowerShell ?

Réponse: Méthode de hashtable qui teste explicitement si une clé existe.

Q89 — EN

What does “ContainsKey” mean or do in PowerShell?

Answer: Hashtable method that explicitly checks whether a key exists.

Q90 — FR

Que signifie / à quoi sert « collection pipeline output » en PowerShell ?

Réponse: Une fonction peut émettre zéro, un ou plusieurs objets; la manière dont le caller reçoit le résultat dépend du nombre d'objets.

Q90 — EN

What does “collection pipeline output” mean or do in PowerShell?

Answer: A function can emit zero, one, or many objects; how the caller receives the result depends on the object count.

10. Opérateurs et conditions / Operators and conditions

Q91 — FR

Que signifie / à quoi sert « -eq / -ne » en PowerShell ?

Réponse: Comparaison égal/différent, par défaut insensible à la casse pour les chaînes.

Q91 — EN

What does “-eq / -ne” mean or do in PowerShell?

Answer: Equality/inequality comparison, case-insensitive by default for strings.

Q92 — FR

Que signifie / à quoi sert « -ceq / -cne » en PowerShell ?

Réponse: Variantes sensibles à la casse.

Q92 — EN

What does “-ceq / -cne” mean or do in PowerShell?

Answer: Case-sensitive comparison variants.

Q93 — FR

Que signifie / à quoi sert « -gt / -ge / -lt / -le » en PowerShell ?

Réponse: Comparaisons numériques ou ordonnables.

Q93 — EN

What does “-gt / -ge / -lt / -le” mean or do in PowerShell?

Answer: Numeric or orderable comparisons.

Q94 — FR

Que signifie / à quoi sert « -like / -notlike » en PowerShell ?

Réponse: Comparaison avec wildcards PowerShell * et ?.

Q94 — EN

What does “-like / -notlike” mean or do in PowerShell?

Answer: Wildcard comparison using PowerShell * and ?.

Q95 — FR

Que signifie / à quoi sert « -match / -notmatch » en PowerShell ?

Réponse: Comparaison par expression régulière; \$Matches reçoit les captures lors d'un match scalaire.

Q95 — EN

What does “-match / -notmatch” mean or do in PowerShell?

Answer: Regular-expression comparison; \$Matches receives captures for scalar matches.

Q96 — FR

Que signifie / à quoi sert « -contains / -in » en PowerShell ?

Réponse: Teste l'appartenance entre collection et valeur, avec l'ordre des opérandes inversé.

Q96 — EN

What does “-contains / -in” mean or do in PowerShell?

Answer: Tests membership between collection and value with reversed operand order.

Q97 — FR

Que signifie / à quoi sert « -and / -or / -not » en PowerShell ?

Réponse: Opérateurs logiques.

Q97 — EN

What does “-and / -or / -not” mean or do in PowerShell?

Answer: Logical operators.

Q98 — FR

Que signifie / à quoi sert « if / elseif / else » en PowerShell ?

Réponse: Sélection conditionnelle principale.

Q98 — EN

What does “if / elseif / else” mean or do in PowerShell?

Answer: Primary conditional selection syntax.

Q99 — FR

Que signifie / à quoi sert « switch » en PowerShell ?

Réponse: Sélection multi-cas puissante, avec options -Regex, -Wildcard et traitement de collections.

Q99 — EN

What does “switch” mean or do in PowerShell?

Answer: Powerful multi-case selection with -Regex, -Wildcard, and collection processing.

Q100 — FR

Que signifie / à quoi sert « ternary ? : » en PowerShell ?

Réponse: Disponible dans PowerShell moderne pour des expressions conditionnelles compactes; éviter l'abus si la lisibilité diminue.

Q100 — EN

What does “ternary ? :” mean or do in PowerShell?

Answer: Available in modern PowerShell for compact conditional expressions; avoid overuse when it hurts readability.

11. Boucles et itération / Loops and iteration

Q101 — FR

Que signifie / à quoi sert « foreach statement » en PowerShell ?

Réponse: Boucle rapide sur une collection déjà en mémoire.

Q101 — EN

What does “foreach statement” mean or do in PowerShell?

Answer: Fast loop over a collection already in memory.

Q102 — FR

Que signifie / à quoi sert « ForEach-Object » en PowerShell ?

Réponse: Traite les objets au fil du pipeline, utile pour streaming et composition de commandes.

Q102 — EN

What does “ForEach-Object” mean or do in PowerShell?

Answer: Processes objects as they flow through the pipeline, useful for streaming and command composition.

Q103 — FR

Que signifie / à quoi sert « for » en PowerShell ?

Réponse: Boucle classique avec initialisation, condition et incrément.

Q103 — EN

What does “for” mean or do in PowerShell?

Answer: Classic loop with initialization, condition, and increment.

Q104 — FR

Que signifie / à quoi sert « while » en PowerShell ?

Réponse: Répète tant qu'une condition est vraie.

Q104 — EN

What does “while” mean or do in PowerShell?

Answer: Repeats while a condition is true.

Q105 — FR

Que signifie / à quoi sert « do while » en PowerShell ?

Réponse: Exécute au moins une fois puis teste la condition.

Q105 — EN

What does “do while” mean or do in PowerShell?

Answer: Executes at least once, then tests the condition.

Q106 — FR

Que signifie / à quoi sert « do until » en PowerShell ?

Réponse: Exécute jusqu'à ce qu'une condition devienne vraie.

Q106 — EN

What does “do until” mean or do in PowerShell?

Answer: Executes until a condition becomes true.

Q107 — FR

Que signifie / à quoi sert « break » en PowerShell ?

Réponse: Quitte la boucle ou le switch courant.

Q107 — EN

What does “break” mean or do in PowerShell?

Answer: Exits the current loop or switch.

Q108 — FR

Que signifie / à quoi sert « continue » en PowerShell ?

Réponse: Passe immédiatement à l'itération suivante.

Q108 — EN

What does “continue” mean or do in PowerShell?

Answer: Immediately continues with the next iteration.

Q109 — FR

Que signifie / à quoi sert « labeled break » en PowerShell ?

Réponse: Permet de quitter une boucle externe nommée dans des boucles imbriquées.

Q109 — EN

What does “labeled break” mean or do in PowerShell?

Answer: Allows exiting a named outer loop in nested loops.

Q110 — FR

Que signifie / à quoi sert « retry loop » en PowerShell ?

Q110 — EN

What does “retry loop” mean or do in PowerShell?

Réponse: Pour hardware/production, une boucle de retry doit avoir un nombre maximum, un timeout et des logs afin de ne pas masquer un défaut intermittent.

Answer: For hardware/production, retry loops should have a maximum count, timeout, and logging so intermittent faults are not hidden.

12. Fonctions et paramètres / Functions and parameters

Q111 — FR

Que signifie / à quoi sert « function » en PowerShell ?

Réponse: Définit une unité réutilisable de logique PowerShell.

Q111 — EN

What does “function” mean or do in PowerShell?

Answer: Defines a reusable unit of PowerShell logic.

Q112 — FR

Que signifie / à quoi sert « param() » en PowerShell ?

Réponse: Déclare les paramètres d'un script ou d'une fonction.

Q112 — EN

What does “param()” mean or do in PowerShell?

Answer: Declares script or function parameters.

Q113 — FR

Que signifie / à quoi sert « [Parameter(Mandatory)] » en PowerShell ?

Réponse: Rend un paramètre obligatoire dans une fonction avancée.

Q113 — EN

What does “[Parameter(Mandatory)]” mean or do in PowerShell?

Answer: Makes a parameter mandatory in an advanced function.

Q114 — FR

Que signifie / à quoi sert « ValidateSet » en PowerShell ?

Réponse: Restreint une valeur à une liste autorisée, utile pour modes Test/Production.

Q114 — EN

What does “ValidateSet” mean or do in PowerShell?

Answer: Restricts a value to an allowed list, useful for Test/Production modes.

Q115 — FR

Que signifie / à quoi sert « ValidateRange » en PowerShell ?

Réponse: Restreint une valeur numérique à une plage, utile pour tension, courant ou timeout.

Q115 — EN

What does “ValidateRange” mean or do in PowerShell?

Answer: Restricts a numeric value to a range, useful for voltage, current, or timeout.

Q116 — FR

Que signifie / à quoi sert « ValidatePattern » en PowerShell ?

Réponse: Valide une chaîne par regex, par exemple un numéro de série.

Q116 — EN

What does “ValidatePattern” mean or do in PowerShell?

Answer: Validates a string with regex, such as a serial number.

Q117 — FR

Que signifie / à quoi sert « default parameter value » en PowerShell ?

Réponse: Valeur utilisée quand l'appelant ne fournit pas le paramètre.

Q117 — EN

What does “default parameter value” mean or do in PowerShell?

Answer: Value used when the caller does not supply a parameter.

Q118 — FR

Que signifie / à quoi sert « return » en PowerShell ?

Réponse: Peut quitter une portée et émettre une valeur, mais tout output non capturé dans la fonction fait également partie du résultat PowerShell.

Q118 — EN

What does “return” mean or do in PowerShell?

Answer: Can exit a scope and emit a value, but any uncaptured output in the function also becomes part of the PowerShell result.

Q119 — FR

Que signifie / à quoi sert « Write-Output vs return » en PowerShell ?

Réponse: Write-Output émet dans le pipeline; return arrête en plus la portée courante après son expression.

Q119 — EN

What does “Write-Output vs return” mean or do in PowerShell?

Answer: Write-Output emits to the pipeline; return additionally exits the current scope after its expression.

Q120 — FR

Que signifie / à quoi sert « single responsibility » en PowerShell ?

Q120 — EN

What does “single responsibility” mean or do in PowerShell?

Réponse: Une fonction de station de test est plus maintenable si elle fait une tâche claire: programmer, mesurer, valider ou logger.

Answer: A test-station function is more maintainable when it has one clear responsibility: program, measure, validate, or log.

13. Fonctions avancées et binding / Advanced functions and binding

Q121 — FR

Que signifie / à quoi sert « [CmdletBinding()] » en PowerShell ?

Réponse: Transforme une fonction en fonction avancée avec comportement proche d'une cmdlet et CommonParameters.

Q121 — EN

What does "[CmdletBinding()]" mean or do in PowerShell?

Answer: Turns a function into an advanced function with cmdlet-like behavior and CommonParameters.

Q122 — FR

Que signifie / à quoi sert « ValueFromPipeline » en PowerShell ?

Réponse: Permet de lier directement les objets du pipeline à un paramètre.

Q122 — EN

What does "ValueFromPipeline" mean or do in PowerShell?

Answer: Allows pipeline objects to bind directly to a parameter.

Q123 — FR

Que signifie / à quoi sert « ValueFromPipelineByPropertyName » en PowerShell ?

Réponse: Lie une propriété d'objet à un paramètre portant le même nom/alias.

Q123 — EN

What does "ValueFromPipelineByPropertyName" mean or do in PowerShell?

Answer: Binds an object property to a parameter with the same name/alias.

Q124 — FR

Que signifie / à quoi sert « begin/process/end » en PowerShell ?

Réponse: Blocs utilisés pour initialiser, traiter chaque objet du pipeline et finaliser.

Q124 — EN

What does "begin/process/end" mean or do in PowerShell?

Answer: Blocks used to initialize, process each pipeline object, and finalize.

Q125 — FR

Que signifie / à quoi sert « SupportsShouldProcess » en PowerShell ?

Réponse: Active -WhatIf/-Confirm via \$PSCmdlet.ShouldProcess pour une fonction qui modifie l'état.

Q125 — EN

What does "SupportsShouldProcess" mean or do in PowerShell?

Answer: Enables -WhatIf/-Confirm via \$PSCmdlet.ShouldProcess for a state-changing function.

Q126 — FR

Que signifie / à quoi sert « \$PSCmdlet » en PowerShell ?

Réponse: Objet de contexte d'une fonction avancée donnant accès à ShouldProcess, parameter set, ThrowTerminatingError, etc.

Q126 — EN

What does "\$PSCmdlet" mean or do in PowerShell?

Answer: Context object in an advanced function exposing ShouldProcess, parameter set, ThrowTerminatingError, and more.

Q127 — FR

Que signifie / à quoi sert « ParameterSetName » en PowerShell ?

Réponse: Permet plusieurs modes d'appel clairement séparés dans une seule fonction.

Q127 — EN

What does "ParameterSetName" mean or do in PowerShell?

Answer: Allows clearly separated invocation modes within one function.

Q128 — FR

Que signifie / à quoi sert « Alias attribute » en PowerShell ?

Réponse: Permet à un paramètre d'accepter d'autres noms, notamment pour pipeline binding.

Q128 — EN

What does "Alias attribute" mean or do in PowerShell?

Answer: Allows a parameter to accept alternate names, including for pipeline binding.

Q129 — FR

Que signifie / à quoi sert « OutputType » en PowerShell ?

Réponse: Documentation indicative du type de sortie attendu; elle ne force pas réellement le type à runtime.

Q129 — EN

What does "OutputType" mean or do in PowerShell?

Answer: Documentation hint for expected output type; it does not actually enforce runtime output type.

Q130 — FR

Que signifie / à quoi sert « comment-based help » en PowerShell ?

Réponse: Commentaires structurés
 .SYNOPSIS/.DESCRIPTION/.PARAMETER/.EXAMPLE exposés par
 Get-Help.

Q130 — EN

What does “comment-based help” mean or do in PowerShell?

Answer: Structured
 .SYNOPSIS/.DESCRIPTION/.PARAMETER/.EXAMPLE comments
 exposed by Get-Help.

14. Scripts, arguments et contexte d'exécution / Scripts, arguments, and execution context

Q131 — FR

Que signifie / à quoi sert « .ps1 » en PowerShell ?

Réponse: Extension standard d'un script PowerShell.

Q131 — EN

What does “.ps1” mean or do in PowerShell?

Answer: Standard PowerShell script extension.

Q132 — FR

Que signifie / à quoi sert « -File » en PowerShell ?

Réponse: Exécute un fichier script depuis powershell.exe/pwsh avec arguments.

Q132 — EN

What does “-File” mean or do in PowerShell?

Answer: Runs a script file from powershell.exe/pwsh with arguments.

Q133 — FR

Que signifie / à quoi sert « -Command » en PowerShell ?

Réponse: Exécute une commande/expression fournie sur la ligne de commande; quoting doit être maîtrisé lorsqu'un autre shell lance PowerShell.

Q133 — EN

What does “-Command” mean or do in PowerShell?

Answer: Runs a supplied command/expression from the command line; quoting must be handled carefully when another shell launches PowerShell.

Q134 — FR

Que signifie / à quoi sert « \$args » en PowerShell ?

Réponse: Tableau des arguments non liés à des paramètres nommés; préférable d'utiliser param() pour les scripts sérieux.

Q134 — EN

What does “\$args” mean or do in PowerShell?

Answer: Array of unbound arguments; param() is preferable for serious scripts.

Q135 — FR

Que signifie / à quoi sert « \$PSScriptRoot » en PowerShell ?

Réponse: Chemin du dossier du script courant, base fiable pour fichiers annexes.

Q135 — EN

What does “\$PSScriptRoot” mean or do in PowerShell?

Answer: Path of the current script directory, a reliable base for accompanying files.

Q136 — FR

Que signifie / à quoi sert « \$PSCommandPath » en PowerShell ?

Réponse: Chemin complet du script courant.

Q136 — EN

What does “\$PSCommandPath” mean or do in PowerShell?

Answer: Full path of the current script.

Q137 — FR

Que signifie / à quoi sert « dot sourcing » en PowerShell ?

Réponse: Charge un script dans la portée courante afin de rendre ses fonctions/variables disponibles. Exemple: . .\Functions.ps1.

Q137 — EN

What does “dot sourcing” mean or do in PowerShell?

Answer: Loads a script into the current scope so its functions/variables remain available. Example: . .\Functions.ps1.

Q138 — FR

Que signifie / à quoi sert « call operator & » en PowerShell ?

Réponse: Exécute un chemin/commande stocké dans une chaîne ou variable. Exemple: & \$toolPath --version.

Q138 — EN

What does “call operator &” mean or do in PowerShell?

Answer: Executes a command/path stored in a string or variable. Example: & \$toolPath --version.

Q139 — FR

Que signifie / à quoi sert « exit code » en PowerShell ?

Réponse: Un script peut appeler exit N pour fournir un code au processus parent; il faut distinguer cela du succès des cmdlets.

Q139 — EN

What does “exit code” mean or do in PowerShell?

Answer: A script can call exit N to return a code to the parent process; this is distinct from cmdlet success.

Q140 — FR**Que signifie / à quoi sert « \$LASTEXITCODE » en PowerShell ?**

Réponse: Code de sortie du dernier programme natif exécuté, très important pour programmer.exe, compiler.exe ou outils de test.

Q140 — EN**What does “\$LASTEXITCODE” mean or do in PowerShell?**

Answer: Exit code from the last native program executed, critical for programmer.exe, compiler.exe, or test tools.

15. Modules et réutilisation / Modules and reuse

Q141 — FR**Que signifie / à quoi sert « .psm1 » en PowerShell ?**

Réponse: Fichier de module script contenant fonctions/variables exportables.

Q141 — EN**What does “.psm1” mean or do in PowerShell?**

Answer: Script module file containing exportable functions/variables.

Q142 — FR**Que signifie / à quoi sert « .psd1 manifest » en PowerShell ?**

Réponse: Manifeste de module décrivant version, auteur, dépendances et éléments exportés.

Q142 — EN**What does “.psd1 manifest” mean or do in PowerShell?**

Answer: Module manifest describing version, author, dependencies, and exports.

Q143 — FR**Que signifie / à quoi sert « Import-Module » en PowerShell ?**

Réponse: Charge un module dans la session. Exemple: Import-Module Pester.

Q143 — EN**What does “Import-Module” mean or do in PowerShell?**

Answer: Loads a module into the session. Example: Import-Module Pester.

Q144 — FR**Que signifie / à quoi sert « Remove-Module » en PowerShell ?**

Réponse: Décharge un module de la session courante.

Q144 — EN**What does “Remove-Module” mean or do in PowerShell?**

Answer: Unloads a module from the current session.

Q145 — FR**Que signifie / à quoi sert « Export-ModuleMember » en PowerShell ?**

Réponse: Contrôle les fonctions/alias/variables exportés par un script module.

Q145 — EN**What does “Export-ModuleMember” mean or do in PowerShell?**

Answer: Controls functions/aliases/variables exported by a script module.

Q146 — FR**Que signifie / à quoi sert « \$env:PSModulePath » en PowerShell ?**

Réponse: Liste des chemins où PowerShell cherche les modules.

Q146 — EN**What does “\$env:PSModulePath” mean or do in PowerShell?**

Answer: List of paths PowerShell searches for modules.

Q147 — FR**Que signifie / à quoi sert « Get-InstalledModule » en PowerShell ?**

Réponse: Liste des modules installés via PowerShellGet, selon la version de PowerShellGet utilisée.

Q147 — EN**What does “Get-InstalledModule” mean or do in PowerShell?**

Answer: Lists modules installed through PowerShellGet, depending on the PowerShellGet version.

Q148 — FR**Que signifie / à quoi sert « version de module » en PowerShell ?**

Réponse: Un environnement de production doit fixer/valider les versions de modules afin d'éviter qu'une mise à jour change le comportement du test.

Q148 — EN**What does “version de module” mean or do in PowerShell?**

Answer: A production environment should pin/validate module versions so updates do not silently change test behavior.

Q149 — FR**Que signifie / à quoi sert « module privé interne » en PowerShell ?**

Réponse: Une équipe hardware peut créer un module interne regroupant flashing, COM, instruments, logging et règles de validation.

Q149 — EN**What does “module privé interne” mean or do in PowerShell?**

Answer: A hardware team can create an internal module that centralizes flashing, COM, instruments, logging, and validation rules.

Q150 — FR

Que signifie / à quoi sert « tests de module » en PowerShell ?

Réponse: Des tests Pester permettent de vérifier les fonctions avant déploiement sur des stations de production.

Q150 — EN

What does “tests de module” mean or do in PowerShell?

Answer: Pester tests can verify module functions before deployment to production stations.

16. Profils et préférences / Profiles and preferences

Q151 — FR

Que signifie / à quoi sert « \$PROFILE » en PowerShell ?

Réponse: Variable indiquant le chemin du profil correspondant à l'hôte/utilisateur courant, avec variantes accessibles comme propriétés.

Q151 — EN

What does “\$PROFILE” mean or do in PowerShell?

Answer: Variable indicating the current host/user profile path, with variants exposed as properties.

Q152 — FR

Que signifie / à quoi sert « profile script » en PowerShell ?

Réponse: Script chargé au démarrage d'un hôte PowerShell; utile pour personnalisation interactive mais dangereux comme dépendance cachée en production.

Q152 — EN

What does “profile script” mean or do in PowerShell?

Answer: Script loaded when a PowerShell host starts; useful interactively but risky as a hidden production dependency.

Q153 — FR

Que signifie / à quoi sert « \$ErrorActionPreference » en PowerShell ?

Réponse: Préférence par défaut pour erreurs non-terminating; Stop est parfois choisi dans un script de test contrôlé.

Q153 — EN

What does “\$ErrorActionPreference” mean or do in PowerShell?

Answer: Default preference for non-terminating errors; Stop is sometimes selected in controlled test scripts.

Q154 — FR

Que signifie / à quoi sert « \$VerbosePreference » en PowerShell ?

Réponse: Contrôle l'affichage de Write-Verbose lorsqu'aucun -Verbose explicite n'est fourni.

Q154 — EN

What does “\$VerbosePreference” mean or do in PowerShell?

Answer: Controls Write-Verbose output when -Verbose is not explicitly supplied.

Q155 — FR

Que signifie / à quoi sert « \$WarningPreference » en PowerShell ?

Réponse: Contrôle le comportement des avertissements.

Q155 — EN

What does “\$WarningPreference” mean or do in PowerShell?

Answer: Controls warning behavior.

Q156 — FR

Que signifie / à quoi sert « \$ProgressPreference » en PowerShell ?

Réponse: Contrôle l'affichage des barres de progression; les désactiver peut accélérer certaines automatisations non interactives.

Q156 — EN

What does “\$ProgressPreference” mean or do in PowerShell?

Answer: Controls progress display; disabling it can speed up some noninteractive automation.

Q157 — FR

Que signifie / à quoi sert « \$ConfirmPreference » en PowerShell ?

Réponse: Influence les demandes de confirmation selon l'impact déclaré des cmdlets.

Q157 — EN

What does “\$ConfirmPreference” mean or do in PowerShell?

Answer: Influences confirmation prompts based on cmdlet-declared impact.

Q158 — FR

Que signifie / à quoi sert « \$PSDefaultParameterValues » en PowerShell ?

Réponse: Hashtable permettant de définir des valeurs de paramètres par défaut de façon centralisée.

Q158 — EN

What does “\$PSDefaultParameterValues” mean or do in PowerShell?

Answer: Hashtable for centrally defining default parameter values.

Q159 — FR

Que signifie / à quoi sert « préférences locales » en PowerShell ?

Réponse: Utiliser une portée locale/script pour modifier temporairement les préférences évite d'affecter la session entière.

Q159 — EN

What does “préférences locales” mean or do in PowerShell?

Answer: Using local/script scope for temporary preference changes avoids affecting the entire session.

Q160 — FR

Que signifie / à quoi sert « -NoProfile en CI/production » en PowerShell ?**Réponse:** Réduit les différences entre postes en évitant le code de profil utilisateur.

Q160 — EN

What does “-NoProfile en CI/production” mean or do in PowerShell?**Answer:** Reduces workstation differences by avoiding user profile code.

17. Gestion des erreurs / Error handling

Q161 — FR

Que signifie / à quoi sert « terminating error » en PowerShell ?**Réponse:** Erreur qui arrête le pipeline/commande et peut être interceptée par catch.

Q161 — EN

What does “terminating error” mean or do in PowerShell?**Answer:** Error that stops the pipeline/command and can be caught by catch.

Q162 — FR

Que signifie / à quoi sert « non-terminating error » en PowerShell ?**Réponse:** Erreur signalée mais la commande peut continuer; - ErrorAction Stop permet souvent de la convertir en erreur terminating pour try/catch.

Q162 — EN

What does “non-terminating error” mean or do in PowerShell?**Answer:** Error that is reported while the command may continue; - ErrorAction Stop often converts it to a terminating error for try/catch.

Q163 — FR

Que signifie / à quoi sert « try/catch/finally » en PowerShell ?**Réponse:** Structure principale pour gérer les erreurs et garantir le nettoyage dans finally.

Q163 — EN

What does “try/catch/finally” mean or do in PowerShell?**Answer:** Primary structure for handling errors and guaranteeing cleanup in finally.

Q164 — FR

Que signifie / à quoi sert « throw » en PowerShell ?**Réponse:** Crée explicitement une erreur terminating.

Q164 — EN

What does “throw” mean or do in PowerShell?**Answer:** Explicitly raises a terminating error.

Q165 — FR

Que signifie / à quoi sert « \$Error » en PowerShell ?**Réponse:** Collection d'erreurs récentes, la plus récente étant généralement à l'index 0.

Q165 — EN

What does “\$Error” mean or do in PowerShell?**Answer:** Collection of recent errors, with the newest generally at index 0.

Q166 — FR

Que signifie / à quoi sert « ErrorRecord » en PowerShell ?**Réponse:** Objet d'erreur contenant Exception, CategoryInfo, FullyQualifiedErrorId, InvocationInfo, etc.

Q166 — EN

What does “ErrorRecord” mean or do in PowerShell?**Answer:** Error object containing Exception, CategoryInfo, FullyQualifiedErrorId, InvocationInfo, and more.

Q167 — FR

Que signifie / à quoi sert « -ErrorVariable » en PowerShell ?**Réponse:** Capture les erreurs d'une commande dans une variable sans perdre la stratégie d'affichage choisie.

Q167 — EN

What does “-ErrorVariable” mean or do in PowerShell?**Answer:** Captures command errors into a variable without losing the chosen display strategy.

Q168 — FR

Que signifie / à quoi sert « \$LASTEXITCODE vs \$? » en PowerShell ?**Réponse:** \$LASTEXITCODE concerne le dernier programme natif; \$? indique le succès de la dernière opération PowerShell, avec nuances selon contexte.

Q168 — EN

What does “\$LASTEXITCODE vs \$?” mean or do in PowerShell?**Answer:** \$LASTEXITCODE applies to the last native program; \$? reports success of the last PowerShell operation, with context-dependent nuances.

Q169 — FR

Que signifie / à quoi sert « cleanup matériel » en PowerShell ?

Q169 — EN

What does “cleanup matériel” mean or do in PowerShell?

Réponse: Un finally peut fermer un port série, remettre une alimentation à zéro ou libérer un fichier même si le test échoue.

Answer: A finally block can close a serial port, reset a power supply, or release a file even when the test fails.

Q170 — FR

Que signifie / à quoi sert « fail fast » en PowerShell ?

Réponse: Dans une station de production, une condition critique doit produire une erreur claire, un code de sortie et un log plutôt que continuer avec des données invalides.

Q170 — EN

What does “fail fast” mean or do in PowerShell?

Answer: In a production station, a critical condition should produce a clear error, exit code, and log rather than continue with invalid data.

18. Streams, output et redirection / Streams, output, and redirection

Q171 — FR

Que signifie / à quoi sert « success output » en PowerShell ?

Réponse: Stream 1 contenant les objets normaux du pipeline.

Q171 — EN

What does “success output” mean or do in PowerShell?

Answer: Stream 1 carrying normal pipeline objects.

Q172 — FR

Que signifie / à quoi sert « error stream » en PowerShell ?

Réponse: Stream 2 contenant les ErrorRecord.

Q172 — EN

What does “error stream” mean or do in PowerShell?

Answer: Stream 2 carrying ErrorRecord objects.

Q173 — FR

Que signifie / à quoi sert « warning stream » en PowerShell ?

Réponse: Stream 3 pour Write-Warning.

Q173 — EN

What does “warning stream” mean or do in PowerShell?

Answer: Stream 3 for Write-Warning.

Q174 — FR

Que signifie / à quoi sert « verbose stream » en PowerShell ?

Réponse: Stream 4 pour Write-Verbose.

Q174 — EN

What does “verbose stream” mean or do in PowerShell?

Answer: Stream 4 for Write-Verbose.

Q175 — FR

Que signifie / à quoi sert « debug stream » en PowerShell ?

Réponse: Stream 5 pour Write-Debug.

Q175 — EN

What does “debug stream” mean or do in PowerShell?

Answer: Stream 5 for Write-Debug.

Q176 — FR

Que signifie / à quoi sert « information stream » en PowerShell ?

Réponse: Stream 6 utilisé notamment par Write-Information et certains outputs d'hôte modernes.

Q176 — EN

What does “information stream” mean or do in PowerShell?

Answer: Stream 6 used by Write-Information and some modern host output.

Q177 — FR

Que signifie / à quoi sert « > / >> » en PowerShell ?

Réponse: Redirige/remplit ou ajoute le success output dans un fichier; pour des objets structurés, Export-Csv/ConvertTo-Json sont souvent meilleurs.

Q177 — EN

What does “> / >>” mean or do in PowerShell?

Answer: Redirects/overwrites or appends success output to a file; structured objects are often better handled with Export-Csv/ConvertTo-Json.

Q178 — FR

Que signifie / à quoi sert « 2> / 2>> » en PowerShell ?

Réponse: Redirige le stream d'erreur.

Q178 — EN

What does “2> / 2>>” mean or do in PowerShell?

Answer: Redirects the error stream.

Q179 — FR

Que signifie / à quoi sert « *>&1 » en PowerShell ?

Réponse: Fusionne tous les streams redirigeables vers le success stream, utile pour capturer un log complet avec prudence.

Q179 — EN

What does “*>&1” mean or do in PowerShell?

Answer: Merges redirectable streams into the success stream, useful for capturing a full log with care.

Q180 — FR

Que signifie / à quoi sert « Write-Host » en PowerShell ?

Réponse: Écrit principalement pour l’affichage utilisateur et ne doit pas remplacer l’émission d’objets lorsqu’un résultat doit être traité par le pipeline.

Q180 — EN

What does “Write-Host” mean or do in PowerShell?

Answer: Writes primarily for user display and should not replace object output when results need pipeline processing.

19. Fichiers, chemins et providers / Files, paths, and providers

Q181 — FR

Que signifie / à quoi sert « Get-ChildItem » en PowerShell ?

Réponse: Liste les éléments d’un provider; sur FileSystem il liste fichiers/dossiers. Exemple: Get-ChildItem C:\Logs -Recurse.

Q181 — EN

What does “Get-ChildItem” mean or do in PowerShell?

Answer: Lists provider items; on FileSystem it lists files/directories. Example: Get-ChildItem C:\Logs -Recurse.

Q182 — FR

Que signifie / à quoi sert « Get-Content » en PowerShell ?

Réponse: Lit le contenu d’un fichier texte, avec options pour tail, raw, encoding, etc.

Q182 — EN

What does “Get-Content” mean or do in PowerShell?

Answer: Reads text file content with options for tail, raw, encoding, and more.

Q183 — FR

Que signifie / à quoi sert « Set-Content » en PowerShell ?

Réponse: Écrit/remplace le contenu d’un fichier ou provider compatible.

Q183 — EN

What does “Set-Content” mean or do in PowerShell?

Answer: Writes/replaces content in a file or compatible provider.

Q184 — FR

Que signifie / à quoi sert « Add-Content » en PowerShell ?

Réponse: Ajoute du contenu à la fin.

Q184 — EN

What does “Add-Content” mean or do in PowerShell?

Answer: Appends content to the end.

Q185 — FR

Que signifie / à quoi sert « Copy-Item » en PowerShell ?

Réponse: Copie éléments du provider.

Q185 — EN

What does “Copy-Item” mean or do in PowerShell?

Answer: Copies provider items.

Q186 — FR

Que signifie / à quoi sert « Move-Item » en PowerShell ?

Réponse: Déplace ou renomme éléments.

Q186 — EN

What does “Move-Item” mean or do in PowerShell?

Answer: Moves or renames items.

Q187 — FR

Que signifie / à quoi sert « Remove-Item » en PowerShell ?

Réponse: Supprime éléments; -Recurse/-Force doivent être utilisés avec prudence.

Q187 — EN

What does “Remove-Item” mean or do in PowerShell?

Answer: Removes items; -Recurse/-Force require care.

Q188 — FR

Que signifie / à quoi sert « Test-Path » en PowerShell ?

Réponse: Teste l’existence/validité d’un chemin.

Q188 — EN

What does “Test-Path” mean or do in PowerShell?

Answer: Tests whether a path exists/is valid.

Q189 — FR

Que signifie / à quoi sert « Resolve-Path » en PowerShell ?

Réponse: Résout un chemin vers son chemin provider complet.

Q189 — EN

What does “Resolve-Path” mean or do in PowerShell?

Answer: Resolves a path to its full provider path.

Q190 — FR

Que signifie / à quoi sert « LiteralPath » en PowerShell ?

Réponse: Traite les caractères *?[] comme littéraux au lieu de wildcards, important pour noms de fichiers inhabituels.

Q190 — EN

What does “LiteralPath” mean or do in PowerShell?

Answer: Treats *?[] characters literally instead of as wildcards, important for unusual file names.

20. CSV, JSON et XML / CSV, JSON, and XML

Q191 — FR

Que signifie / à quoi sert « Import-Csv » en PowerShell ?

Réponse: Convertit les lignes CSV en objets avec propriétés basées sur les en-têtes.

Q191 — EN

What does “Import-Csv” mean or do in PowerShell?

Answer: Converts CSV rows into objects with properties based on headers.

Q192 — FR

Que signifie / à quoi sert « Export-Csv » en PowerShell ?

Réponse: Sérialise des objets en CSV, pratique pour rapports de test tabulaires. Exemple: Export-Csv report.csv -NoTypeInfoInformation.

Q192 — EN

What does “Export-Csv” mean or do in PowerShell?

Answer: Serializes objects to CSV, useful for tabular test reports. Example: Export-Csv report.csv -NoTypeInfoInformation.

Q193 — FR

Que signifie / à quoi sert « ConvertTo-Json » en PowerShell ?

Réponse: Convertit un objet en JSON; -Depth doit parfois être augmenté pour structures imbriquées.

Q193 — EN

What does “ConvertTo-Json” mean or do in PowerShell?

Answer: Converts an object to JSON; -Depth sometimes must be increased for nested structures.

Q194 — FR

Que signifie / à quoi sert « ConvertFrom-Json » en PowerShell ?

Réponse: Convertit JSON en objets PowerShell.

Q194 — EN

What does “ConvertFrom-Json” mean or do in PowerShell?

Answer: Converts JSON into PowerShell objects.

Q195 — FR

Que signifie / à quoi sert « Invoke-RestMethod + JSON » en PowerShell ?

Réponse: Les APIs REST renvoient souvent du JSON que PowerShell transforme directement en objets.

Q195 — EN

What does “Invoke-RestMethod + JSON” mean or do in PowerShell?

Answer: REST APIs often return JSON that PowerShell turns directly into objects.

Q196 — FR

Que signifie / à quoi sert « [xml] cast » en PowerShell ?

Réponse: Charge une chaîne/du contenu XML en XmlDocument pour navigation par nœuds/propriétés.

Q196 — EN

What does “[xml] cast” mean or do in PowerShell?

Answer: Loads XML text/content into an XmlDocument for node/property navigation.

Q197 — FR

Que signifie / à quoi sert « Select-Xml » en PowerShell ?

Réponse: Exécute des requêtes XPath sur XML.

Q197 — EN

What does “Select-Xml” mean or do in PowerShell?

Answer: Runs XPath queries against XML.

Q198 — FR

Que signifie / à quoi sert « encoding » en PowerShell ?

Réponse: Pour échanger des fichiers avec des outils de production, spécifier/valider l'encodage évite les incompatibilités.

Q198 — EN

What does “encoding” mean or do in PowerShell?

Answer: When exchanging files with production tools, specifying/validating encoding prevents incompatibilities.

Q199 — FR

Que signifie / à quoi sert « culture invariant CSV » en PowerShell ?

Réponse: Les séparateurs/décimales dépendent parfois de la locale; définir un format contractuel entre machines.

Q199 — EN

What does “culture invariant CSV” mean or do in PowerShell?

Answer: Separators/decimals can depend on locale; define a contractual machine-to-machine format.

Q200 — FR

Que signifie / à quoi sert « objet de résultat » en PowerShell ?

Réponse: Créer un [pscustomobject] par test puis Export-Csv/ConvertTo-Json sépare proprement données et présentation.

Q200 — EN

What does “objet de résultat” mean or do in PowerShell?

Answer: Creating one [pscustomobject] per test and exporting it separates data from presentation cleanly.

21. Texte et expressions régulières / Text and regular expressions

Q201 — FR Que signifie / à quoi sert « Select-String » en PowerShell ? Réponse: Recherche du texte/regex dans fichiers ou chaînes, similaire à grep mais retourne des MatchInfo structurés.	Q201 — EN What does “Select-String” mean or do in PowerShell? Answer: Searches text/regex in files or strings, similar to grep but returns structured MatchInfo objects.
Q202 — FR Que signifie / à quoi sert « -split » en PowerShell ? Réponse: Sépare une chaîne selon un motif regex.	Q202 — EN What does “-split” mean or do in PowerShell? Answer: Splits a string using a regex pattern.
Q203 — FR Que signifie / à quoi sert « -join » en PowerShell ? Réponse: Joint une collection en une chaîne avec un séparateur.	Q203 — EN What does “-join” mean or do in PowerShell? Answer: Joins a collection into a string with a separator.
Q204 — FR Que signifie / à quoi sert « Replace() method » en PowerShell ? Réponse: Remplacement littéral via méthode .NET String.Replace.	Q204 — EN What does “Replace() method” mean or do in PowerShell? Answer: Literal replacement using the .NET String.Replace method.
Q205 — FR Que signifie / à quoi sert « -replace » en PowerShell ? Réponse: Remplacement regex PowerShell.	Q205 — EN What does “-replace” mean or do in PowerShell? Answer: PowerShell regex replacement.
Q206 — FR Que signifie / à quoi sert « [regex]::Escape » en PowerShell ? Réponse: Échappe une chaîne pour l'insérer littéralement dans une regex.	Q206 — EN What does “[regex]::Escape” mean or do in PowerShell? Answer: Escapes a string for literal insertion into a regex.
Q207 — FR Que signifie / à quoi sert « \$Matches » en PowerShell ? Réponse: Hashtable automatique contenant les captures du dernier -match scalaire réussi.	Q207 — EN What does “\$Matches” mean or do in PowerShell? Answer: Automatic hashtable containing captures from the last successful scalar -match.
Q208 — FR Que signifie / à quoi sert « named capture » en PowerShell ? Réponse: Regex (?<name>...) permettant de récupérer une donnée par nom dans \$Matches.	Q208 — EN What does “named capture” mean or do in PowerShell? Answer: Regex (?<name>...) allows retrieving captured data by name from \$Matches.
Q209 — FR Que signifie / à quoi sert « parsing vs structured data » en PowerShell ? Réponse: Si un outil peut fournir JSON/CSV, préférer cela au parsing fragile de texte libre.	Q209 — EN What does “parsing vs structured data” mean or do in PowerShell? Answer: If a tool can provide JSON/CSV, prefer that over fragile free-text parsing.
Q210 — FR Que signifie / à quoi sert « validation numéro de série » en PowerShell ? Réponse: Une regex peut vérifier format/longueur avant d'envoyer le SN à un programmeur ou serveur.	Q210 — EN What does “validation numéro de série” mean or do in PowerShell? Answer: A regex can validate serial-number format/length before sending it to a programmer or server.

22. Processus, services et tâches planifiées / Processes, services, and scheduled tasks

Q211 — FR

Que signifie / à quoi sert « Get-Process » en PowerShell ?

Réponse: Liste les processus et expose PID, CPU, mémoire, handles, etc.

Q211 — EN

What does “Get-Process” mean or do in PowerShell?

Answer: Lists processes and exposes PID, CPU, memory, handles, and more.

Q212 — FR

Que signifie / à quoi sert « Start-Process » en PowerShell ?

Réponse: Lance un exécutable avec arguments, working directory, elevation ou attente selon options.

Q212 — EN

What does “Start-Process” mean or do in PowerShell?

Answer: Starts an executable with arguments, working directory, elevation, or waiting depending on options.

Q213 — FR

Que signifie / à quoi sert « -Wait » en PowerShell ?

Réponse: Avec Start-Process, attend la fin du processus; pour récupérer facilement un exit code, utiliser aussi -PassThru.

Q213 — EN

What does “-Wait” mean or do in PowerShell?

Answer: With Start-Process, waits for process completion; use -PassThru as well to easily read the exit code.

Q214 — FR

Que signifie / à quoi sert « Stop-Process » en PowerShell ?

Réponse: Arrête un processus par nom/ID; à utiliser avec prudence en production.

Q214 — EN

What does “Stop-Process” mean or do in PowerShell?

Answer: Stops a process by name/ID; use carefully in production.

Q215 — FR

Que signifie / à quoi sert « Get-Service » en PowerShell ?

Réponse: Liste les services Windows et leur état.

Q215 — EN

What does “Get-Service” mean or do in PowerShell?

Answer: Lists Windows services and their state.

Q216 — FR

Que signifie / à quoi sert « Start-Service / Stop-Service » en PowerShell ?

Réponse: Démarre/arrête un service Windows.

Q216 — EN

What does “Start-Service / Stop-Service” mean or do in PowerShell?

Answer: Starts/stops a Windows service.

Q217 — FR

Que signifie / à quoi sert « Restart-Service » en PowerShell ?

Réponse: Redémarre un service, utile après configuration de drivers/outils si documenté.

Q217 — EN

What does “Restart-Service” mean or do in PowerShell?

Answer: Restarts a service, useful after documented driver/tool configuration.

Q218 — FR

Que signifie / à quoi sert « Get-ScheduledTask » en PowerShell ?

Réponse: Liste les tâches planifiées via le module ScheduledTasks sous Windows.

Q218 — EN

What does “Get-ScheduledTask” mean or do in PowerShell?

Answer: Lists scheduled tasks via the Windows ScheduledTasks module.

Q219 — FR

Que signifie / à quoi sert « Start-ScheduledTask » en PowerShell ?

Réponse: Déclenche une tâche existante.

Q219 — EN

What does “Start-ScheduledTask” mean or do in PowerShell?

Answer: Starts an existing scheduled task.

Q220 — FR

Que signifie / à quoi sert « Register-ScheduledTask » en PowerShell ?

Réponse: Crée une tâche planifiée; utile pour maintenance/collecte, mais une station de test doit documenter exactement ce qui s'exécute automatiquement.

Q220 — EN

What does “Register-ScheduledTask” mean or do in PowerShell?

Answer: Creates a scheduled task; useful for maintenance/collection, but test stations must document exactly what runs automatically.

23. Registre et environnement Windows / Windows Registry and environment

Q221 — FR

Que signifie / à quoi sert « Registry provider » en PowerShell ?

Réponse: Expose HKLM: et HKCU: comme lecteurs PowerShell afin d'utiliser Get-Item/Get-ItemProperty.

Q221 — EN

What does “Registry provider” mean or do in PowerShell?

Answer: Exposes HKLM: and HKCU: as PowerShell drives so Get-Item/Get-ItemProperty can be used.

Q222 — FR

Que signifie / à quoi sert « Get-ItemProperty » en PowerShell ?

Réponse: Lit les valeurs d'une clé de registre ou propriétés d'un provider.

Q222 — EN

What does “Get-ItemProperty” mean or do in PowerShell?

Answer: Reads registry values or properties from a provider item.

Q223 — FR

Que signifie / à quoi sert « Set-ItemProperty » en PowerShell ?

Réponse: Crée/modifie une propriété/valeur de registre.

Q223 — EN

What does “Set-ItemProperty” mean or do in PowerShell?

Answer: Creates/modifies a registry property/value.

Q224 — FR

Que signifie / à quoi sert « New-ItemProperty » en PowerShell ?

Réponse: Crée explicitement une nouvelle valeur de registre avec type.

Q224 — EN

What does “New-ItemProperty” mean or do in PowerShell?

Answer: Explicitly creates a new registry value with a type.

Q225 — FR

Que signifie / à quoi sert « Remove-ItemProperty » en PowerShell ?

Réponse: Supprime une valeur de registre.

Q225 — EN

What does “Remove-ItemProperty” mean or do in PowerShell?

Answer: Deletes a registry value.

Q226 — FR

Que signifie / à quoi sert « \$env:NAME » en PowerShell ?

Réponse: Accède à une variable d'environnement du processus courant.

Q226 — EN

What does “\$env:NAME” mean or do in PowerShell?

Answer: Accesses an environment variable in the current process.

Q227 — FR

Que signifie / à quoi sert « [Environment]::SetEnvironmentVariable » en PowerShell ?

Réponse: Modifie une variable au niveau Process/User/Machine selon l'overload utilisé.

Q227 — EN

What does “[Environment]::SetEnvironmentVariable” mean or do in PowerShell?

Answer: Changes an environment variable at Process/User/Machine scope depending on the overload used.

Q228 — FR

Que signifie / à quoi sert « \$env:PATH » en PowerShell ?

Réponse: Liste de chemins d'exécutables; la modifier temporairement pour une toolchain est souvent préférable à une modification machine globale.

Q228 — EN

What does “\$env:PATH” mean or do in PowerShell?

Answer: Executable search path; temporarily changing it for a toolchain is often preferable to modifying the machine globally.

Q229 — FR

Que signifie / à quoi sert « vue registre 32/64 bits » en PowerShell ?

Réponse: Un outil 32 bits et un outil 64 bits peuvent voir des vues différentes de certaines clés; vérifier l'architecture de l'hôte.

Q229 — EN

What does “vue registre 32/64 bits” mean or do in PowerShell?

Answer: A 32-bit and 64-bit tool can see different views of some registry keys; verify host architecture.

Q230 — FR

Que signifie / à quoi sert « backup avant modification » en PowerShell ?

Réponse: Une automatisation qui modifie le registre doit lire/loguer l'ancienne valeur et idéalement fournir un rollback.

Q230 — EN

What does “backup avant modification” mean or do in PowerShell?

Answer: Automation that changes the registry should read/log the old value and ideally provide rollback.

24. Réseau et diagnostics TCP/IP / Networking and TCP/IP diagnostics

Q231 — FR

Que signifie / à quoi sert « Get-NetIPAddress » en PowerShell ?

Réponse: Liste les adresses IP configurées sous Windows.

Q231 — EN

What does “Get-NetIPAddress” mean or do in PowerShell?

Answer: Lists configured IP addresses on Windows.

Q232 — FR

Que signifie / à quoi sert « Get-NetAdapter » en PowerShell ?

Réponse: Liste les interfaces réseau, état, vitesse et MAC.

Q232 — EN

What does “Get-NetAdapter” mean or do in PowerShell?

Answer: Lists network adapters, status, speed, and MAC address.

Q233 — FR

Que signifie / à quoi sert « Get-NetIPConfiguration » en PowerShell ?

Réponse: Vue synthétique interface/IP/gateway/DNS.

Q233 — EN

What does “Get-NetIPConfiguration” mean or do in PowerShell?

Answer: Summary view of interface/IP/gateway/DNS configuration.

Q234 — FR

Que signifie / à quoi sert « Test-Connection » en PowerShell ?

Réponse: Teste la connectivité ICMP et retourne des objets structurés.

Q234 — EN

What does “Test-Connection” mean or do in PowerShell?

Answer: Tests ICMP connectivity and returns structured objects.

Q235 — FR

Que signifie / à quoi sert « Test-NetConnection » en PowerShell ?

Réponse: Teste connectivité et notamment un port TCP, très utile pour diagnostiquer un device Ethernet. Exemple: Test-NetConnection 192.168.1.20 -Port 502.

Q235 — EN

What does “Test-NetConnection” mean or do in PowerShell?

Answer: Tests connectivity including TCP ports, useful for diagnosing Ethernet devices. Example: Test-NetConnection 192.168.1.20 -Port 502.

Q236 — FR

Que signifie / à quoi sert « Resolve-DnsName » en PowerShell ?

Réponse: Résout DNS avec résultats structurés sous Windows.

Q236 — EN

What does “Resolve-DnsName” mean or do in PowerShell?

Answer: Resolves DNS with structured results on Windows.

Q237 — FR

Que signifie / à quoi sert « Get-NetTCPConnection » en PowerShell ?

Réponse: Liste connexions TCP et ports d'écoute avec PID.

Q237 — EN

What does “Get-NetTCPConnection” mean or do in PowerShell?

Answer: Lists TCP connections and listening ports with PID.

Q238 — FR

Que signifie / à quoi sert « Get-NetRoute » en PowerShell ?

Réponse: Affiche la table de routage.

Q238 — EN

What does “Get-NetRoute” mean or do in PowerShell?

Answer: Displays the routing table.

Q239 — FR

Que signifie / à quoi sert « Get-NetNeighbor » en PowerShell ?

Réponse: Affiche voisins IP/ARP/ND selon plateforme/module.

Q239 — EN

What does “Get-NetNeighbor” mean or do in PowerShell?

Answer: Displays IP neighbors/ARP/ND depending on platform/module.

Q240 — FR

Que signifie / à quoi sert « timeout réseau » en PowerShell ?

Réponse: Un test hardware réseau doit distinguer DNS, reachability, TCP connect, protocole applicatif et timeout afin de localiser correctement la panne.

Q240 — EN

What does “timeout réseau” mean or do in PowerShell?

Answer: A network hardware test should separate DNS, reachability, TCP connect, application protocol, and timeout to localize failures correctly.

25. Web, HTTP et API REST / Web, HTTP, and REST APIs

Q241 — FR

Que signifie / à quoi sert « Invoke-WebRequest » en PowerShell ?

Q241 — EN

What does “Invoke-WebRequest” mean or do in PowerShell?

Réponse: Effectue des requêtes HTTP et retourne une réponse web structurée/du contenu; utile pour téléchargement et diagnostic.

Answer: Performs HTTP requests and returns a structured web response/content; useful for downloads and diagnostics.

Q242 — FR

Que signifie / à quoi sert « Invoke-RestMethod » en PowerShell ?

Réponse: Optimisé pour APIs REST et convertit souvent JSON/XML en objets automatiquement.

Q242 — EN

What does “Invoke-RestMethod” mean or do in PowerShell?

Answer: Optimized for REST APIs and often converts JSON/XML into objects automatically.

Q243 — FR

Que signifie / à quoi sert « -Method » en PowerShell ?

Réponse: Sélectionne GET, POST, PUT, DELETE, PATCH, etc.

Q243 — EN

What does “-Method” mean or do in PowerShell?

Answer: Selects GET, POST, PUT, DELETE, PATCH, and more.

Q244 — FR

Que signifie / à quoi sert « -Headers » en PowerShell ?

Réponse: Ajoute des en-têtes HTTP, par exemple Authorization ou identifiant de station.

Q244 — EN

What does “-Headers” mean or do in PowerShell?

Answer: Adds HTTP headers such as Authorization or station identity.

Q245 — FR

Que signifie / à quoi sert « -Body » en PowerShell ?

Réponse: Fournit le corps d'une requête; pour JSON, utiliser ConvertTo-Json et ContentType approprié.

Q245 — EN

What does “-Body” mean or do in PowerShell?

Answer: Supplies request body; for JSON, use ConvertTo-Json and the appropriate ContentType.

Q246 — FR

Que signifie / à quoi sert « -ContentType » en PowerShell ?

Réponse: Déclare le media type envoyé, par exemple application/json.

Q246 — EN

What does “-ContentType” mean or do in PowerShell?

Answer: Declares the media type being sent, such as application/json.

Q247 — FR

Que signifie / à quoi sert « HTTP status » en PowerShell ?

Réponse: Une automatisation doit vérifier les erreurs/status et ne pas supposer qu'une réponse réseau signifie que la transaction métier a réussi.

Q247 — EN

What does “HTTP status” mean or do in PowerShell?

Answer: Automation should check errors/status and not assume that receiving a network response means the business transaction succeeded.

Q248 — FR

Que signifie / à quoi sert « TLS/certificats » en PowerShell ?

Réponse: Ne pas désactiver la validation TLS dans une station de production sans justification; corriger plutôt la chaîne de certificats.

Q248 — EN

What does “TLS/certificats” mean or do in PowerShell?

Answer: Do not disable TLS validation in production without justification; fix the certificate chain instead.

Q249 — FR

Que signifie / à quoi sert « upload résultats » en PowerShell ?

Réponse: Une station peut envoyer un objet de résultat sérialisé en JSON vers MES/serveur de qualité.

Q249 — EN

What does “upload résultats” mean or do in PowerShell?

Answer: A station can serialize a result object to JSON and send it to a MES/quality server.

Q250 — FR

Que signifie / à quoi sert « retry HTTP » en PowerShell ?

Réponse: Les retries doivent être limités et adaptés au type d'erreur; répéter aveuglément un POST peut dupliquer une opération.

Q250 — EN

What does “retry HTTP” mean or do in PowerShell?

Answer: Retries must be bounded and tailored to the error type; blindly retrying a POST can duplicate an operation.

26. WMI et CIM / WMI and CIM

Q251 — FR

Que signifie / à quoi sert « Get-CimInstance » en PowerShell ?

Réponse: Interroge les classes CIM/WMI modernes, généralement préféré à Get-WmiObject pour les nouveaux scripts.

Q251 — EN

What does “Get-CimInstance” mean or do in PowerShell?

Answer: Queries modern CIM/WMI classes, generally preferred over Get-WmiObject for new scripts.

Q252 — FR

Que signifie / à quoi sert « Get-WmiObject » en PowerShell ?

Réponse: Cmdlet historique de Windows PowerShell; absente de PowerShell 7 et remplacée par CIM pour les nouveaux développements.

Q252 — EN

What does “Get-WmiObject” mean or do in PowerShell?

Answer: Legacy Windows PowerShell cmdlet; absent from PowerShell 7 and replaced by CIM for new development.

Q253 — FR

Que signifie / à quoi sert « Win32_OperatingSystem » en PowerShell ?

Réponse: Classe fournissant informations OS, boot time, mémoire, etc.

Q253 — EN

What does “Win32_OperatingSystem” mean or do in PowerShell?

Answer: Class providing OS information, boot time, memory, and more.

Q254 — FR

Que signifie / à quoi sert « Win32_ComputerSystem » en PowerShell ?

Réponse: Informations machine telles que modèle, fabricant, mémoire totale.

Q254 — EN

What does “Win32_ComputerSystem” mean or do in PowerShell?

Answer: Machine information such as model, manufacturer, and total memory.

Q255 — FR

Que signifie / à quoi sert « Win32_BIOS » en PowerShell ?

Réponse: Informations BIOS/firmware exposées par WMI.

Q255 — EN

What does “Win32_BIOS” mean or do in PowerShell?

Answer: BIOS/firmware information exposed by WMI.

Q256 — FR

Que signifie / à quoi sert « Win32_PnPEntity » en PowerShell ?

Réponse: Informations générales sur périphériques Plug and Play, utile en diagnostic hardware.

Q256 — EN

What does “Win32_PnPEntity” mean or do in PowerShell?

Answer: General Plug and Play device information, useful for hardware diagnostics.

Q257 — FR

Que signifie / à quoi sert « Win32_SerialPort » en PowerShell ?

Réponse: Peut exposer certains ports série matériels, mais ne liste pas toujours tous les ports virtuels USB-série; compléter avec PnP/registry/.NET.

Q257 — EN

What does “Win32_SerialPort” mean or do in PowerShell?

Answer: Can expose some hardware serial ports but may not list every virtual USB-serial port; supplement with PnP/registry/.NET.

Q258 — FR

Que signifie / à quoi sert « CimSession » en PowerShell ?

Réponse: Session réutilisable vers une machine locale/distante pour plusieurs requêtes CIM.

Q258 — EN

What does “CimSession” mean or do in PowerShell?

Answer: Reusable local/remote session for multiple CIM queries.

Q259 — FR

Que signifie / à quoi sert « filter server-side » en PowerShell ?

Réponse: -Filter réduit le volume retourné en filtrant côté provider, souvent plus efficace que Where-Object après récupération massive.

Q259 — EN

What does “filter server-side” mean or do in PowerShell?

Answer: -Filter reduces returned data by filtering at the provider, often more efficient than Where-Object after retrieving everything.

Q260 — FR

Que signifie / à quoi sert « méthodes CIM » en PowerShell ?

Réponse: Invoke-CimMethod appelle une méthode d'une classe/instance CIM lorsque nécessaire.

Q260 — EN

What does “méthodes CIM” mean or do in PowerShell?

Answer: Invoke-CimMethod calls a CIM class/instance method when needed.

27. PnP, USB et Hardware IDs / PnP, USB, and hardware IDs

Q261 — FR

Que signifie / à quoi sert « Get-PnpDevice » en PowerShell ?

Q261 — EN

What does “Get-PnpDevice” mean or do in PowerShell?

Réponse: Liste les périphériques Plug and Play via le module Windows PnpDevice.

Answer: Lists Plug and Play devices through the Windows PnpDevice module.

Q262 — FR

Que signifie / à quoi sert « Get-PnpDeviceProperty » en PowerShell ?

Réponse: Lit des propriétés PnP comme Hardware IDs, bus, manufacturer ou location selon la propriété demandée.

Q262 — EN

What does “Get-PnpDeviceProperty” mean or do in PowerShell?

Answer: Reads PnP properties such as hardware IDs, bus, manufacturer, or location depending on requested property.

Q263 — FR

Que signifie / à quoi sert « Instanceld » en PowerShell ?

Réponse: Identifiant PnP d'une instance de périphérique; utile pour corréler un device USB précis.

Q263 — EN

What does “Instanceld” mean or do in PowerShell?

Answer: PnP instance identifier for a device instance; useful for correlating a specific USB device.

Q264 — FR

Que signifie / à quoi sert « VID/PID » en PowerShell ?

Réponse: Vendor ID et Product ID USB apparaissent généralement dans les Hardware IDs sous forme VID_xxxx&PID_xxxx.

Q264 — EN

What does “VID/PID” mean or do in PowerShell?

Answer: USB Vendor ID and Product ID generally appear in hardware IDs as VID_xxxx&PID_xxxx.

Q265 — FR

Que signifie / à quoi sert « serial USB » en PowerShell ?

Réponse: Certains devices USB exposent un numéro de série dans l'Instanceld, mais cela dépend du firmware/des descripteurs USB.

Q265 — EN

What does “serial USB” mean or do in PowerShell?

Answer: Some USB devices expose a serial number in Instanceld, depending on firmware/USB descriptors.

Q266 — FR

Que signifie / à quoi sert « status PnP » en PowerShell ?

Réponse: Un périphérique peut être présent mais en erreur driver/configuration; vérifier Status, ProblemCode ou événements associés.

Q266 — EN

What does “status PnP” mean or do in PowerShell?

Answer: A device can be present yet have a driver/configuration error; check Status, ProblemCode, or associated events.

Q267 — FR

Que signifie / à quoi sert « pnputil.exe depuis PowerShell » en PowerShell ?

Réponse: PowerShell peut orchestrer l'outil natif PnPUtl et vérifier \$LASTEXITCODE pour installer/énumérer des drivers selon droits/politique.

Q267 — EN

What does “pnputil.exe depuis PowerShell” mean or do in PowerShell?

Answer: PowerShell can orchestrate native PnPUtl and inspect \$LASTEXITCODE to install/enumerate drivers according to permissions/policy.

Q268 — FR

Que signifie / à quoi sert « driver package » en PowerShell ?

Réponse: Pour une station de test, valider la version du driver requise par le fixture/device avant de lancer le test.

Q268 — EN

What does “driver package” mean or do in PowerShell?

Answer: For a test station, validate the required driver version for the fixture/device before starting tests.

Q269 — FR

Que signifie / à quoi sert « hot-plug detection » en PowerShell ?

Réponse: Un script peut poller PnP ou utiliser des événements WMI/CIM/.NET, mais il faut gérer race conditions et retrait du device.

Q269 — EN

What does “hot-plug detection” mean or do in PowerShell?

Answer: A script can poll PnP or use WMI/CIM/.NET events, but must handle race conditions and device removal.

Q270 — FR

Que signifie / à quoi sert « USB topology » en PowerShell ?

Réponse: Pour certains diagnostics, l'emplacement/port/hub est important; les propriétés PnP peuvent aider mais la topologie complète peut nécessiter des API/outils spécialisés.

Q270 — EN

What does “USB topology” mean or do in PowerShell?

Answer: For some diagnostics, physical port/hub location matters; PnP properties can help but full topology may require specialized APIs/tools.

28. Ports série et communication COM / Serial ports and COM communication

Q271 — FR

Que signifie / à quoi sert « `[System.IO.Ports.SerialPort]` » en PowerShell ?

Réponse: Classe .NET courante pour ouvrir et communiquer avec un port COM depuis PowerShell.

Q271 — EN

What does “[System.IO.Ports.SerialPort]” mean or do in PowerShell?

Answer: Common .NET class for opening and communicating with a COM port from PowerShell.

Q272 — FR

Que signifie / à quoi sert « `GetPortNames()` » en PowerShell ?

Réponse: Méthode statique listant les noms COM vus par .NET. Exemple: `[IO.Ports.SerialPort]::GetPortNames()`.

Q272 — EN

What does “GetPortNames()” mean or do in PowerShell?

Answer: Static method listing COM port names visible to .NET. Example: `[IO.Ports.SerialPort]::GetPortNames()`.

Q273 — FR

Que signifie / à quoi sert « `BaudRate` » en PowerShell ?

Réponse: Vitesse symboles/seconde configurée, qui doit correspondre au périphérique.

Q273 — EN

What does “BaudRate” mean or do in PowerShell?

Answer: Configured symbol rate, which must match the device.

Q274 — FR

Que signifie / à quoi sert « `Parity/DataBits/StopBits` » en PowerShell ?

Réponse: Paramètres de framing UART qui doivent correspondre aux deux extrémités.

Q274 — EN

What does “Parity/DataBits/StopBits” mean or do in PowerShell?

Answer: UART framing parameters that must match at both ends.

Q275 — FR

Que signifie / à quoi sert « `ReadTimeout/WriteTimeout` » en PowerShell ?

Réponse: Empêchent un script de rester bloqué indéfiniment lors d'une absence de réponse.

Q275 — EN

What does “ReadTimeout/WriteTimeout” mean or do in PowerShell?

Answer: Prevent a script from blocking indefinitely when a device does not respond.

Q276 — FR

Que signifie / à quoi sert « `Open()/Close()` » en PowerShell ?

Réponse: Ouvre/ferme le handle du port; Close doit être garanti même en cas d'erreur.

Q276 — EN

What does “Open()/Close()” mean or do in PowerShell?

Answer: Opens/closes the port handle; Close should be guaranteed even when errors occur.

Q277 — FR

Que signifie / à quoi sert « `WriteLine()/ReadLine()` » en PowerShell ?

Réponse: Pratiques pour protocoles texte terminés par une nouvelle ligne; inadaptés aux trames binaires arbitraires sans adaptation.

Q277 — EN

What does “WriteLine()/ReadLine()” mean or do in PowerShell?

Answer: Convenient for newline-terminated text protocols; not suitable for arbitrary binary frames without adaptation.

Q278 — FR

Que signifie / à quoi sert « `Read()/Write()` bytes » en PowerShell ?

Réponse: Permettent protocoles binaires avec buffers `byte[]`.

Q278 — EN

What does “Read()/Write() bytes” mean or do in PowerShell?

Answer: Support binary protocols using `byte[]` buffers.

Q279 — FR

Que signifie / à quoi sert « `DTR/RTS` » en PowerShell ?

Réponse: Lignes de contrôle pouvant servir au handshake ou au reset/boot de certains devices; leur rôle dépend du hardware.

Q279 — EN

What does “DTR/RTS” mean or do in PowerShell?

Answer: Control lines used for handshaking or reset/boot on some devices; behavior depends on hardware.

Q280 — FR

Que signifie / à quoi sert « `flush/buffer` » en PowerShell ?

Réponse: Il faut comprendre les buffers d'entrée/sortie et vider/attendre correctement pour éviter de confondre ancienne donnée et nouvelle réponse.

Q280 — EN

What does “flush/buffer” mean or do in PowerShell?

Answer: Input/output buffers must be handled carefully so stale data is not mistaken for a new response.

29. Journaux Windows et diagnostic système / Windows logs and system diagnostics

Q281 — FR

Que signifie / à quoi sert « Get-WinEvent » en PowerShell ?

Réponse: Cmdlet moderne pour lire efficacement Event Logs et fichiers .evtx.

Q281 — EN

What does “Get-WinEvent” mean or do in PowerShell?

Answer: Modern cmdlet for efficiently reading Event Logs and .evtx files.

Q282 — FR

Que signifie / à quoi sert « FilterHashtable » en PowerShell ?

Réponse: Filtre côté API par LogName, Id, Level, ProviderName, StartTime, etc., plus performant qu'un Where-Object massif.

Q282 — EN

What does “FilterHashtable” mean or do in PowerShell?

Answer: Filters at the API by LogName, Id, Level, ProviderName, StartTime, and more, more efficient than large Where-Object filters.

Q283 — FR

Que signifie / à quoi sert « Event ID » en PowerShell ?

Réponse: Identifiant d'événement propre au provider; il faut l'interpréter avec le provider et le contexte.

Q283 — EN

What does “Event ID” mean or do in PowerShell?

Answer: Event identifier specific to the provider; interpret it together with provider and context.

Q284 — FR

Que signifie / à quoi sert « System log » en PowerShell ?

Réponse: Contient notamment événements de drivers, services, power et kernel.

Q284 — EN

What does “System log” mean or do in PowerShell?

Answer: Contains driver, service, power, and kernel events among others.

Q285 — FR

Que signifie / à quoi sert « Application log » en PowerShell ?

Réponse: Contient des événements d'applications/services selon leurs providers.

Q285 — EN

What does “Application log” mean or do in PowerShell?

Answer: Contains application/service events depending on their providers.

Q286 — FR

Que signifie / à quoi sert « Microsoft-Windows-* Operational » en PowerShell ?

Réponse: De nombreux composants modernes écrivent dans des journaux Operational dédiés plus utiles que System/Application.

Q286 — EN

What does “Microsoft-Windows-* Operational” mean or do in PowerShell?

Answer: Many modern components write to dedicated Operational logs that can be more useful than System/Application.

Q287 — FR

Que signifie / à quoi sert « Export-Clixml » en PowerShell ?

Réponse: Peut sérialiser des objets PowerShell pour diagnostic interne, avec limites sur objets live/méthodes.

Q287 — EN

What does “Export-Clixml” mean or do in PowerShell?

Answer: Can serialize PowerShell objects for internal diagnostics, with limitations on live objects/methods.

Q288 — FR

Que signifie / à quoi sert « Start-Transcript » en PowerShell ?

Réponse: Enregistre une transcription de session hôte; utile pour support mais ce n'est pas un substitut à un log structuré de test.

Q288 — EN

What does “Start-Transcript” mean or do in PowerShell?

Answer: Records a host session transcript; useful for support but not a substitute for structured test logging.

Q289 — FR

Que signifie / à quoi sert « timestamp ISO 8601 » en PowerShell ?

Réponse: Format de temps non ambigu recommandé pour logs, par exemple yyyy-MM-ddTHH:mm:ss.fffK.

Q289 — EN

What does “timestamp ISO 8601” mean or do in PowerShell?

Answer: Unambiguous time format recommended for logs, such as yyyy-MM-ddTHH:mm:ss.fffK.

Q290 — FR

Que signifie / à quoi sert « corrélation » en PowerShell ?

Réponse: Un diagnostic solide corrèle timestamp du test, serial number, firmware, station ID et événements Windows.

Q290 — EN

What does “corrélation” mean or do in PowerShell?

Answer: Strong diagnostics correlate test timestamp, serial number, firmware, station ID, and Windows events.

30. Remoting, jobs et parallélisme / Remoting, jobs, and parallelism

Q291 — FR

Que signifie / à quoi sert « Invoke-Command » en PowerShell ?

Réponse: Exécute du code dans une session distante ou locale selon configuration.

Q291 — EN

What does “Invoke-Command” mean or do in PowerShell?

Answer: Runs code in a remote or local session depending on configuration.

Q292 — FR

Que signifie / à quoi sert « New-PSSession » en PowerShell ?

Réponse: Crée une session persistante réutilisable pour plusieurs commandes distantes.

Q292 — EN

What does “New-PSSession” mean or do in PowerShell?

Answer: Creates a persistent session reusable for multiple remote commands.

Q293 — FR

Que signifie / à quoi sert « Enter-PSSession » en PowerShell ?

Réponse: Ouvre une session distante interactive.

Q293 — EN

What does “Enter-PSSession” mean or do in PowerShell?

Answer: Opens an interactive remote session.

Q294 — FR

Que signifie / à quoi sert « WinRM » en PowerShell ?

Réponse: Transport historique principal de PowerShell Remoting sous Windows; nécessite configuration/authentification correctes.

Q294 — EN

What does “WinRM” mean or do in PowerShell?

Answer: Traditional main transport for PowerShell Remoting on Windows; requires proper configuration/authentication.

Q295 — FR

Que signifie / à quoi sert « SSH remoting » en PowerShell ?

Réponse: PowerShell moderne peut utiliser SSH comme transport de remoting selon configuration.

Q295 — EN

What does “SSH remoting” mean or do in PowerShell?

Answer: Modern PowerShell can use SSH as a remoting transport when configured.

Q296 — FR

Que signifie / à quoi sert « Start-Job » en PowerShell ?

Réponse: Lance un background job dans un processus séparé selon l'implémentation Windows/PowerShell.

Q296 — EN

What does “Start-Job” mean or do in PowerShell?

Answer: Starts a background job, generally in a separate process depending on implementation.

Q297 — FR

Que signifie / à quoi sert « Receive-Job » en PowerShell ?

Réponse: Récupère l'output d'un job.

Q297 — EN

What does “Receive-Job” mean or do in PowerShell?

Answer: Retrieves job output.

Q298 — FR

Que signifie / à quoi sert « ForEach-Object -Parallel » en PowerShell ?

Réponse: PowerShell 7 peut traiter des éléments en parallèle avec runspaces; utile mais nécessite de gérer thread safety et accès matériel exclusif.

Q298 — EN

What does “ForEach-Object -Parallel” mean or do in PowerShell?

Answer: PowerShell 7 can process items in parallel using runspaces; useful but requires thread-safety and exclusive hardware-access management.

Q299 — FR

Que signifie / à quoi sert « \$using: » en PowerShell ?

Réponse: Permet de référencer certaines variables locales dans code distant/job/parallel selon contexte.

Q299 — EN

What does “\$using:” mean or do in PowerShell?

Answer: Allows referencing certain local variables inside remote/job/parallel code depending on context.

Q300 — FR

Que signifie / à quoi sert « ne pas paralléliser un fixture unique » en PowerShell ?

Réponse: Deux threads ne doivent pas commander simultanément le même port COM, programmer ou instrument sauf si l'architecture le prévoit explicitement.

Q300 — EN

What does “ne pas paralléliser un fixture unique” mean or do in PowerShell?

Answer: Two threads should not simultaneously control the same COM port, programmer, or instrument unless the architecture explicitly supports it.

31. Sécurité, execution policy et credentials / Security, execution policy, and credentials

Q301 — FR

Que signifie / à quoi sert « ExecutionPolicy » en PowerShell ?

Réponse: Mécanisme de sécurité opérationnelle qui contrôle notamment le chargement de scripts; ce n'est pas une frontière de sécurité absolue.

Q301 — EN

What does "ExecutionPolicy" mean or do in PowerShell?

Answer: Operational safety mechanism controlling script loading among other things; it is not an absolute security boundary.

Q302 — FR

Que signifie / à quoi sert « Get-ExecutionPolicy -List » en PowerShell ?

Réponse: Affiche les politiques par scope (MachinePolicy, UserPolicy, Process, CurrentUser, LocalMachine).

Q302 — EN

What does "Get-ExecutionPolicy -List" mean or do in PowerShell?

Answer: Shows policies by scope (MachinePolicy, UserPolicy, Process, CurrentUser, LocalMachine).

Q303 — FR

Que signifie / à quoi sert « RemoteSigned » en PowerShell ?

Réponse: Politique courante exigeant notamment une signature pour certains scripts marqués comme provenant d'Internet.

Q303 — EN

What does "RemoteSigned" mean or do in PowerShell?

Answer: Common policy that requires a signature for certain scripts marked as originating from the Internet.

Q304 — FR

Que signifie / à quoi sert « Unblock-File » en PowerShell ?

Réponse: Retire le Mark-of-the-Web d'un fichier lorsqu'on a validé sa provenance.

Q304 — EN

What does "Unblock-File" mean or do in PowerShell?

Answer: Removes Mark-of-the-Web from a file after its origin has been validated.

Q305 — FR

Que signifie / à quoi sert « signature de script » en PowerShell ?

Réponse: Authenticode permet de signer et vérifier l'intégrité/provenance de scripts selon politique/certificat.

Q305 — EN

What does "signature de script" mean or do in PowerShell?

Answer: Authenticode can sign and verify script integrity/origin according to policy/certificate.

Q306 — FR

Que signifie / à quoi sert « Get-AuthenticodeSignature » en PowerShell ?

Réponse: Vérifie l'état de signature d'un script/fichier.

Q306 — EN

What does "Get-AuthenticodeSignature" mean or do in PowerShell?

Answer: Checks a script/file Authenticode signature status.

Q307 — FR

Que signifie / à quoi sert « PSCredential » en PowerShell ?

Réponse: Objet contenant nom utilisateur et SecureString pour mot de passe; utile avec cmdlets qui acceptent explicitement un credential.

Q307 — EN

What does "PSCredential" mean or do in PowerShell?

Answer: Object containing username and SecureString password for cmdlets that explicitly accept a credential.

Q308 — FR

Que signifie / à quoi sert « Get-Credential » en PowerShell ?

Réponse: Demande un credential via prompt et retourne PSCredential.

Q308 — EN

What does "Get-Credential" mean or do in PowerShell?

Answer: Prompts for a credential and returns a PSCredential.

Q309 — FR

Que signifie / à quoi sert « secret en clair » en PowerShell ?

Réponse: Ne jamais stocker directement un mot de passe/API key en clair dans un script ou repo.

Q309 — EN

What does "secret en clair" mean or do in PowerShell?

Answer: Never store passwords/API keys in plaintext inside a script or repository.

Q310 — FR

Que signifie / à quoi sert « least privilege » en PowerShell ?

Réponse: Une station de test doit fonctionner avec le minimum de privilèges; n'élever que les opérations qui l'exigent.

Q310 — EN

What does "least privilege" mean or do in PowerShell?

Answer: A test station should run with least privilege and elevate only operations that require it.

32. PowerShell ISE, VS Code et debugging / PowerShell ISE, VS Code, and debugging

Q311 — FR

Que signifie / à quoi sert « breakpoint » en PowerShell ?

Réponse: Arrête l'exécution à une ligne, commande ou variable afin d'inspecter l'état.

Q311 — EN

What does “breakpoint” mean or do in PowerShell?

Answer: Pauses execution at a line, command, or variable so state can be inspected.

Q312 — FR

Que signifie / à quoi sert « Set-PSBreakpoint » en PowerShell ?

Réponse: Crée des breakpoints PowerShell programmatiquement.

Q312 — EN

What does “Set-PSBreakpoint” mean or do in PowerShell?

Answer: Creates PowerShell breakpoints programmatically.

Q313 — FR

Que signifie / à quoi sert « Get-PSBreakpoint » en PowerShell ?

Réponse: Liste les breakpoints actifs.

Q313 — EN

What does “Get-PSBreakpoint” mean or do in PowerShell?

Answer: Lists active breakpoints.

Q314 — FR

Que signifie / à quoi sert « Remove-PSBreakpoint » en PowerShell ?

Réponse: Supprime des breakpoints.

Q314 — EN

What does “Remove-PSBreakpoint” mean or do in PowerShell?

Answer: Removes breakpoints.

Q315 — FR

Que signifie / à quoi sert « step into/over/out » en PowerShell ?

Réponse: Commandes de debugging qui contrôlent l'exécution des fonctions appelées.

Q315 — EN

What does “step into/over/out” mean or do in PowerShell?

Answer: Debugging actions controlling execution through called functions.

Q316 — FR

Que signifie / à quoi sert « watch variables » en PowerShell ?

Réponse: ISE/VS Code permettent d'inspecter variables et expressions pendant un arrêt.

Q316 — EN

What does “watch variables” mean or do in PowerShell?

Answer: ISE/VS Code allow inspecting variables and expressions while paused.

Q317 — FR

Que signifie / à quoi sert « PowerShell ISE limitations » en PowerShell ?

Réponse: ISE cible Windows PowerShell; il ne faut pas valider un script PowerShell 7 uniquement dans ISE.

Q317 — EN

What does “PowerShell ISE limitations” mean or do in PowerShell?

Answer: ISE targets Windows PowerShell; a PowerShell 7 script should not be validated only in ISE.

Q318 — FR

Que signifie / à quoi sert « VS Code launch configuration » en PowerShell ?

Réponse: launch.json/extension settings peuvent définir le fichier, arguments et environnement de debug selon la configuration.

Q318 — EN

What does “VS Code launch configuration” mean or do in PowerShell?

Answer: launch.json/extension settings can define file, arguments, and debug environment depending on setup.

Q319 — FR

Que signifie / à quoi sert « PSScriptAnalyzer » en PowerShell ?

Réponse: Analyse statique de style, compatibilité et erreurs potentielles via règles configurables.

Q319 — EN

What does “PSScriptAnalyzer” mean or do in PowerShell?

Answer: Static analysis for style, compatibility, and potential issues through configurable rules.

Q320 — FR

Que signifie / à quoi sert « debug hardware » en PowerShell ?

Réponse: Pendant le debug d'un script hardware, il faut protéger l'équipement: un breakpoint peut laisser un relais, une alimentation ou un port dans un état actif.

Q320 — EN

What does “debug hardware” mean or do in PowerShell?

Answer: While debugging hardware scripts, protect the equipment: a breakpoint can leave a relay, supply, or port in an active state.

33. Git, build et toolchains / Git, build, and toolchains

Q321 — FR

Que signifie / à quoi sert « git.exe depuis PowerShell » en PowerShell ?

Réponse: PowerShell peut orchestrer Git comme programme natif et vérifier \$LASTEXITCODE.

Q321 — EN

What does “git.exe depuis PowerShell” mean or do in PowerShell?

Answer: PowerShell can orchestrate Git as a native program and inspect \$LASTEXITCODE.

Q322 — FR

Que signifie / à quoi sert « dotnet CLI » en PowerShell ?

Réponse: PowerShell peut lancer dotnet build/test/publish et transformer les résultats en pipeline de build.

Q322 — EN

What does “dotnet CLI” mean or do in PowerShell?

Answer: PowerShell can run dotnet build/test/publish and turn results into a build pipeline.

Q323 — FR

Que signifie / à quoi sert « cmake » en PowerShell ?

Réponse: PowerShell peut préparer des répertoires, lancer CMake/Ninja et archiver les artefacts.

Q323 — EN

What does “cmake” mean or do in PowerShell?

Answer: PowerShell can prepare directories, run CMake/Ninja, and archive artifacts.

Q324 — FR

Que signifie / à quoi sert « MSBuild » en PowerShell ?

Réponse: Peut être lancé depuis PowerShell avec arguments explicitement quotés et exit code contrôlé.

Q324 — EN

What does “MSBuild” mean or do in PowerShell?

Answer: Can be launched from PowerShell with explicitly quoted arguments and controlled exit code.

Q325 — FR

Que signifie / à quoi sert « Python » en PowerShell ?

Réponse: Un script PowerShell peut appeler Python pour calculs/GUI/outils spécialisés et échanger via JSON/CSV/exit codes.

Q325 — EN

What does “Python” mean or do in PowerShell?

Answer: A PowerShell script can call Python for calculations/GUI/specialized tools and exchange data through JSON/CSV/exit codes.

Q326 — FR

Que signifie / à quoi sert « C#.NET assembly » en PowerShell ?

Réponse: PowerShell peut charger des assemblies .NET avec Add-Type ou utiliser directement des classes disponibles.

Q326 — EN

What does “C#.NET assembly” mean or do in PowerShell?

Answer: PowerShell can load .NET assemblies with Add-Type or directly use available classes.

Q327 — FR

Que signifie / à quoi sert « artifact version » en PowerShell ?

Réponse: Inclure commit Git, version firmware et version de script dans le rapport améliore fortement la traçabilité.

Q327 — EN

What does “artifact version” mean or do in PowerShell?

Answer: Including Git commit, firmware version, and script version in reports greatly improves traceability.

Q328 — FR

Que signifie / à quoi sert « reproducible build » en PowerShell ?

Réponse: Une automation robuste fixe versions/outils et évite de dépendre de PATH/profils non contrôlés.

Q328 — EN

What does “reproducible build” mean or do in PowerShell?

Answer: Robust automation pins tools/versions and avoids uncontrolled PATH/profile dependencies.

Q329 — FR

Que signifie / à quoi sert « native argument quoting » en PowerShell ?

Réponse: Les règles de quoting vers un programme natif peuvent différer selon Windows/PowerShell/version; tester les chemins avec espaces et arguments spéciaux.

Q329 — EN

What does “native argument quoting” mean or do in PowerShell?

Answer: Quoting rules for native programs can differ by Windows/PowerShell/version; test paths with spaces and special arguments.

Q330 — FR

Que signifie / à quoi sert « CI integration » en PowerShell ?

Réponse: PowerShell peut servir de couche d'orchestration dans GitHub Actions, Azure DevOps ou autres CI, tout en conservant des fonctions réutilisables localement.

Q330 — EN

What does “CI integration” mean or do in PowerShell?

Answer: PowerShell can serve as an orchestration layer in GitHub Actions, Azure DevOps, or other CI while keeping reusable local functions.

34. Automatisation hardware et instruments / Hardware and instrument automation

Q331 — FR

Que signifie / à quoi sert « orchestration » en PowerShell ?

Réponse: PowerShell est particulièrement adapté pour enchaîner programmer, alimentation, instrument, fichiers, réseau et rapport dans un workflow de test.

Q331 — EN

What does “orchestration” mean or do in PowerShell?

Answer: PowerShell is particularly well suited to orchestrate programmers, power supplies, instruments, files, networking, and reporting in a test workflow.

Q332 — FR

Que signifie / à quoi sert « SCPI over TCP/serial » en PowerShell ?

Réponse: Des instruments peuvent être pilotés par chaînes SCPI via TCP, VISA/assemblies, USBTMC ou série selon leurs interfaces.

Q332 — EN

What does “SCPI over TCP/serial” mean or do in PowerShell?

Answer: Instruments can be controlled with SCPI strings over TCP, VISA/assemblies, USBTMC, or serial depending on interfaces.

Q333 — FR

Que signifie / à quoi sert « timeout instrument » en PowerShell ?

Réponse: Chaque commande instrument doit avoir un timeout explicite et une gestion des réponses incomplètes.

Q333 — EN

What does “timeout instrument” mean or do in PowerShell?

Answer: Each instrument command should have an explicit timeout and incomplete-response handling.

Q334 — FR

Que signifie / à quoi sert « measurement object » en PowerShell ?

Réponse: Retourner un objet avec Name, Value, Unit, Min, Max, Pass plutôt qu'une phrase rend le test exploitable automatiquement.

Q334 — EN

What does “measurement object” mean or do in PowerShell?

Answer: Returning an object with Name, Value, Unit, Min, Max, Pass instead of a sentence makes the test machine-processable.

Q335 — FR

Que signifie / à quoi sert « limits configuration » en PowerShell ?

Réponse: Stocker les limites dans un fichier versionné JSON/CSV plutôt que dispersées dans le code facilite revue et maintenance.

Q335 — EN

What does “limits configuration” mean or do in PowerShell?

Answer: Storing limits in a versioned JSON/CSV file instead of scattering them through code improves review and maintenance.

Q336 — FR

Que signifie / à quoi sert « fixture detection » en PowerShell ?

Réponse: Avant le test, vérifier présence du programmer, COM port, USB fixture et instruments évite des faux FAIL produit.

Q336 — EN

What does “fixture detection” mean or do in PowerShell?

Answer: Before testing, verify programmer, COM port, USB fixture, and instrument presence to avoid false product failures.

Q337 — FR

Que signifie / à quoi sert « safe state » en PowerShell ?

Réponse: Au démarrage et en finally, mettre sorties/relais/alimentations dans un état sûr.

Q337 — EN

What does “safe state” mean or do in PowerShell?

Answer: At startup and in finally, place outputs/relays/power supplies in a safe state.

Q338 — FR

Que signifie / à quoi sert « calibration status » en PowerShell ?

Réponse: Une station sérieuse vérifie date/version de calibration des instruments avant d'accepter une mesure.

Q338 — EN

What does “calibration status” mean or do in PowerShell?

Answer: A serious station verifies instrument calibration date/version before accepting measurements.

Q339 — FR

Que signifie / à quoi sert « raw data » en PowerShell ?

Réponse: Conserver si possible données brutes pertinentes facilite analyse des défauts après production.

Q339 — EN

What does “raw data” mean or do in PowerShell?

Answer: Keeping relevant raw data where practical helps analyze failures after production.

Q340 — FR

Que signifie / à quoi sert « hardware abstraction » en PowerShell ?

Q340 — EN

What does “hardware abstraction” mean or do in PowerShell?

Réponse: Créer des fonctions comme Set-SupplyVoltage ou Read-DmmVoltage masque les détails du protocole et permet de changer d'instrument plus facilement.

Answer: Functions such as Set-SupplyVoltage or Read-DmmVoltage hide protocol details and make instrument replacement easier.

35. Firmware flashing et programmation / Firmware flashing and programming

Q341 — FR

Que signifie / à quoi sert « programme natif » en PowerShell ?

Réponse: Les outils STM32CubeProgrammer, MPLAB IPE/CLI, J-Link, OpenOCD ou vendor tools peuvent être orchestrés depuis PowerShell selon leur CLI.

Q341 — EN

What does “programme natif” mean or do in PowerShell?

Answer: STM32CubeProgrammer, MPLAB IPE/CLI, J-Link, OpenOCD, or vendor tools can be orchestrated from PowerShell through their CLIs.

Q342 — FR

Que signifie / à quoi sert « \$LASTEXITCODE après flashing » en PowerShell ?

Réponse: Capturer immédiatement le code de l'outil natif avant d'exécuter une autre commande.

Q342 — EN

What does “\$LASTEXITCODE après flashing” mean or do in PowerShell?

Answer: Capture the native tool exit code immediately before running another command.

Q343 — FR

Que signifie / à quoi sert « verify after program » en PowerShell ?

Réponse: Une séquence robuste programme puis vérifie, ou utilise l'option de verify fournie par l'outil.

Q343 — EN

What does “verify after program” mean or do in PowerShell?

Answer: A robust sequence programs and then verifies, or uses the tool's built-in verify option.

Q344 — FR

Que signifie / à quoi sert « device identity » en PowerShell ?

Réponse: Lire device ID/serial/chip ID avant programmation peut éviter de flasher le mauvais produit.

Q344 — EN

What does “device identity” mean or do in PowerShell?

Answer: Reading device ID/serial/chip ID before programming can prevent flashing the wrong product.

Q345 — FR

Que signifie / à quoi sert « firmware hash » en PowerShell ?

Réponse: Vérifier SHA-256 de l'image avant usage réduit le risque de fichier corrompu ou mauvais artefact.

Q345 — EN

What does “firmware hash” mean or do in PowerShell?

Answer: Checking the firmware image SHA-256 before use reduces the risk of a corrupted/wrong artifact.

Q346 — FR

Que signifie / à quoi sert « firmware version » en PowerShell ?

Réponse: Après flashing, lire la version depuis device ou log de l'outil afin de confirmer l'image réellement active.

Q346 — EN

What does “firmware version” mean or do in PowerShell?

Answer: After flashing, read the version from the device or tool log to confirm the active image.

Q347 — FR

Que signifie / à quoi sert « erase policy » en PowerShell ?

Réponse: Full erase vs sector erase doit suivre la stratégie produit, notamment si calibration/serial sont conservés dans Flash.

Q347 — EN

What does “erase policy” mean or do in PowerShell?

Answer: Full erase versus sector erase must follow product policy, especially when calibration/serial data is stored in Flash.

Q348 — FR

Que signifie / à quoi sert « programmer connection retry » en PowerShell ?

Réponse: Un retry limité peut gérer une connexion transitoire, mais il faut distinguer mauvais contact fixture d'un device réellement défectueux.

Q348 — EN

What does “programmer connection retry” mean or do in PowerShell?

Answer: A bounded retry can handle transient connection issues, but must distinguish fixture contact problems from a truly defective device.

Q349 — FR

Que signifie / à quoi sert « logs du programmer » en PowerShell ?

Réponse: Conserver stdout/stderr et exit code facilite le diagnostic d'échecs SWD/JTAG/ICSP.

Q349 — EN

What does “logs du programmer” mean or do in PowerShell?

Answer: Keeping programmer stdout/stderr and exit code helps diagnose SWD/JTAG/ICSP failures.

Q350 — FR

Que signifie / à quoi sert « tool version pinning » en PowerShell ?

Réponse: Une mise à jour du programmer peut changer options/algorithmes; valider et enregistrer la version utilisée en production.

Q350 — EN

What does “tool version pinning” mean or do in PowerShell?

Answer: A programmer update can change options/algorithms; validate and record the version used in production.

36. Stations de test et traçabilité production / Production test stations and traceability

Q351 — FR

Que signifie / à quoi sert « serial number » en PowerShell ?

Réponse: Identifiant produit doit être validé, unique et associé à chaque résultat.

Q351 — EN

What does “serial number” mean or do in PowerShell?

Answer: Product identifier should be validated, unique, and associated with every result.

Q352 — FR

Que signifie / à quoi sert « station ID » en PowerShell ?

Réponse: Identifie la station/fixture ayant produit les mesures.

Q352 — EN

What does “station ID” mean or do in PowerShell?

Answer: Identifies the station/fixture that produced the measurements.

Q353 — FR

Que signifie / à quoi sert « operator ID » en PowerShell ?

Réponse: Selon processus, associe l'opérateur au test sans exposer plus de données que nécessaire.

Q353 — EN

What does “operator ID” mean or do in PowerShell?

Answer: Depending on process, associates the operator with the test without exposing unnecessary data.

Q354 — FR

Que signifie / à quoi sert « PASS/FAIL déterministe » en PowerShell ?

Réponse: Les critères doivent être numériques/logiques et versionnés plutôt que dépendre d'un jugement manuel ambigu.

Q354 — EN

What does “PASS/FAIL déterministe” mean or do in PowerShell?

Answer: Criteria should be numeric/logical and versioned rather than depend on ambiguous manual judgment.

Q355 — FR

Que signifie / à quoi sert « test sequence version » en PowerShell ?

Réponse: Rapport doit inclure version du script/module/limits pour reproduire une décision.

Q355 — EN

What does “test sequence version” mean or do in PowerShell?

Answer: Report should include script/module/limits version to reproduce a decision.

Q356 — FR

Que signifie / à quoi sert « timestamp » en PowerShell ?

Réponse: Utiliser un timestamp non ambigu et idéalement synchroniser l'horloge des stations.

Q356 — EN

What does “timestamp” mean or do in PowerShell?

Answer: Use an unambiguous timestamp and ideally synchronize station clocks.

Q357 — FR

Que signifie / à quoi sert « MES/database upload » en PowerShell ?

Réponse: Envoyer résultats seulement après validation locale et gérer les pannes réseau sans perdre la traçabilité.

Q357 — EN

What does “MES/database upload” mean or do in PowerShell?

Answer: Upload results only after local validation and handle network outages without losing traceability.

Q358 — FR

Que signifie / à quoi sert « local queue » en PowerShell ?

Réponse: Une file locale peut conserver temporairement les résultats si le serveur est indisponible, puis les réessayer avec idempotence.

Q358 — EN

What does “local queue” mean or do in PowerShell?

Answer: A local queue can temporarily retain results when the server is unavailable and retry them idempotently.

Q359 — FR

Que signifie / à quoi sert « golden unit » en PowerShell ?

Q359 — EN

What does “golden unit” mean or do in PowerShell?

Réponse: Une unité de référence connue peut aider à vérifier quotidiennement fixture/instruments, mais ne remplace pas calibration/maintenance.

Answer: A known-good golden unit can help verify fixture/instruments daily but does not replace calibration/maintenance.

Q360 — FR

Que signifie / à quoi sert « audit trail » en PowerShell ?

Réponse: Ne jamais écraser silencieusement un FAIL précédent; conserver historique des retests selon la politique qualité.

Q360 — EN

What does “audit trail” mean or do in PowerShell?

Answer: Never silently overwrite a previous FAIL; keep retest history according to quality policy.

37. Analyse de code - pipeline et objets / Code analysis - pipeline and objects

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q361 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 51 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

\$busy

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q362 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 52 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

\$busy

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q363 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 53 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

\$busy

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q364 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 54 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q365 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 55 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q366 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 56 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q367 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 57 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q368 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 58 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q369 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 59 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q370 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 60 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q371 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 61 |
    Sort-Object CPU -Descending |
```

```
Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q372 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 62 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q373 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 63 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q374 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 64 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU
```

```
$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q375 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 65 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q376 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 66 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q377 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 67 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q378 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 68 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q379 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 69 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

Q380 — Code

```
$busy = Get-Process |
    Where-Object CPU -gt 70 |
    Sort-Object CPU -Descending |
    Select-Object -First 3 Name, Id, CPU

$busy
```

Question FR: Expliquez ce pipeline et le type de données contenu dans \$busy.

Question EN: Explain this pipeline and the type of data stored in \$busy.

Réponse: Get-Process produit des objets Process. Where-Object filtre selon CPU, Sort-Object trie, puis Select-Object garde trois éléments et seulement les propriétés demandées. \$busy contient encore des objets structurés, pas du texte formaté.

Answer: Get-Process emits Process objects. Where-Object filters on CPU, Sort-Object orders them, and Select-Object keeps three items with the requested properties. \$busy still contains structured objects, not formatted text.

38. Analyse de code - collections et résultats structurés / Code analysis - collections and structured results

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q381 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.1 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_1.csv -NoTypeInfoation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q382 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.2 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_2.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q383 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.3 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_3.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q384 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.4 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_4.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q385 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.5 + $n/10), 3)
        Pass    = $true
    }
}
```

```
}
$results | Export-Csv .\result_5.csv -NoTypeInfoInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q386 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.6 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_6.csv -NoTypeInfoInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q387 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.7 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_7.csv -NoTypeInfoInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q388 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.8 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_8.csv -NoTypeInfoInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q389 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((0.9 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_9.csv -NoTypeInfoation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q390 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.0 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_10.csv -NoTypeInfoation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q391 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.1 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_11.csv -NoTypeInfoation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q392 — Code

```
$results = foreach ($n in 1..3) {
```

```
[pscustomobject]@{
    Channel = $n
    Voltage = [math]::Round((1.2 + $n/10), 3)
    Pass    = $true
}
}
$results | Export-Csv .\result_12.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q393 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.3 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_13.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q394 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.4 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_14.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q395 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.5 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_15.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q396 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.6 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_16.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q397 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.7 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_17.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q398 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.8 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_18.csv -NoTypeInformation
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q399 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((1.9 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_19.csv -NoTypeInfo
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

Q400 — Code

```
$results = foreach ($n in 1..3) {
    [pscustomobject]@{
        Channel = $n
        Voltage = [math]::Round((2.0 + $n/10), 3)
        Pass    = $true
    }
}
$results | Export-Csv .\result_20.csv -NoTypeInfo
```

Question FR: Pourquoi [pscustomobject] est-il préférable à des lignes de texte pour ce rapport ?

Réponse: Chaque mesure devient un objet avec propriétés Channel, Voltage et Pass. Export-Csv peut alors créer des colonnes propres, et le même objet peut être filtré, envoyé en JSON ou analysé sans reparsing du texte.

Question EN: Why is [pscustomobject] preferable to text lines for this report?

Answer: Each measurement becomes an object with Channel, Voltage, and Pass properties. Export-Csv can create clean columns, and the same object can be filtered, serialized to JSON, or analyzed without reparsing text.

39. Analyse de code - fichiers et chemins / Code analysis - files and paths

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q401 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_1.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q402 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
```



```
New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_2.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q403 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_3.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q404 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_4.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q405 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_5.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q406 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_6.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q407 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_7.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q408 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_8.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q409 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_9.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q410 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_10.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q411 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_11.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q412 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_12.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q413 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_13.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q414 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_14.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q415 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_15.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q416 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_16.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q417 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
```

```
}
$log = Join-Path $logDir 'station_17.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q418 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_18.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q419 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_19.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

Q420 — Code

```
$logDir = Join-Path $PSScriptRoot 'logs'
if (-not (Test-Path $logDir)) {
    New-Item -ItemType Directory -Path $logDir | Out-Null
}
$log = Join-Path $logDir 'station_20.log'
"Start $(Get-Date -Format o)" | Set-Content -Path $log
```

Question FR: Que fait ce code et pourquoi utilise-t-il \$PSScriptRoot et Join-Path ?

Réponse: Il crée un dossier logs à côté du script s'il n'existe pas, construit un chemin de log et écrit un timestamp ISO. \$PSScriptRoot rend le package indépendant du répertoire courant et Join-Path construit le chemin proprement.

Question EN: What does this code do, and why does it use \$PSScriptRoot and Join-Path?

Answer: It creates a logs folder beside the script if needed, builds a log path, and writes an ISO timestamp. \$PSScriptRoot makes the package independent of the current directory, and Join-Path builds the path safely.

40. Analyse de code - texte, regex et parsing / Code analysis - text, regex, and parsing

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q421 — Code

```
$line = 'SN=AB0001;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q422 — Code

```
$line = 'SN=AB0002;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q423 — Code

```
$line = 'SN=AB0003;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q424 — Code

```
$line = 'SN=AB0004;V=3.298;I=0.124'
```

```
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q425 — Code

```
$line = 'SN=AB0005;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q426 — Code

```
$line = 'SN=AB0006;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q427 — Code

```
$line = 'SN=AB0007;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
}
```



```

    }
} else {
    throw 'Invalid frame'
}

```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q428 — Code

```

$line = 'SN=AB0008;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}

```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q429 — Code

```

$line = 'SN=AB0009;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}

```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q430 — Code

```

$line = 'SN=AB0010;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}

```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q431 — Code

```
$line = 'SN=AB0011;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q432 — Code

```
$line = 'SN=AB0012;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q433 — Code

```
$line = 'SN=AB0013;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.**Question EN: Explain the parsing and the role of named captures.**

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q434 — Code

```
$line = 'SN=AB0014;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q435 — Code

```
$line = 'SN=AB0015;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q436 — Code

```
$line = 'SN=AB0016;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q437 — Code

```
$line = 'SN=AB0017;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q438 — Code

```
$line = 'SN=AB0018;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q439 — Code

```
$line = 'SN=AB0019;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}
```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute trame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

Q440 — Code

```
$line = 'SN=AB0020;V=3.298;I=0.124'
if ($line -match '^SN=(?<sn>[^\;]+);V=(?<v>[0-9.]+);I=(?<i>[0-9.]+)$') {
    [pscustomobject]@{
        Serial = $Matches.sn
    }
}
```

```

        Voltage = [double]$Matches.v
        Current = [double]$Matches.i
    }
} else {
    throw 'Invalid frame'
}

```

Question FR: Expliquez le parsing et le rôle des captures nommées.

Question EN: Explain the parsing and the role of named captures.

Réponse: La regex exige un format complet et capture SN, V et I sous des noms explicites. \$Matches fournit les captures, qui sont converties en nombres avant de construire l'objet. Toute frame différente provoque une erreur au lieu d'être acceptée silencieusement.

Answer: The regex requires the full expected format and captures SN, V, and I under explicit names. \$Matches exposes the captures, which are converted to numbers before building the object. Any different frame throws an error instead of being silently accepted.

41. Analyse de code - erreurs et nettoyage / Code analysis - errors and cleanup

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q441 — Code

```

$port = [IO.Ports.SerialPort]::new('COM4',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 11
} finally {
    if ($port.IsOpen) { $port.Close() }
}

```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q442 — Code

```

$port = [IO.Ports.SerialPort]::new('COM5',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 12
} finally {
    if ($port.IsOpen) { $port.Close() }
}

```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q443 — Code

```
$port = [IO.Ports.SerialPort]::new('COM6',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 13
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q444 — Code

```
$port = [IO.Ports.SerialPort]::new('COM7',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 14
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q445 — Code

```
$port = [IO.Ports.SerialPort]::new('COM8',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 10
} finally {
```

```
if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q446 — Code

```
$port = [IO.Ports.SerialPort]::new('COM9',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 11
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q447 — Code

```
$port = [IO.Ports.SerialPort]::new('COM3',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 12
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q448 — Code

```
$port = [IO.Ports.SerialPort]::new('COM4',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
```



```

$reply = $port.ReadLine()
if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 13
} finally {
    if ($port.IsOpen) { $port.Close() }
}

```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q449 — Code

```

$port = [IO.Ports.SerialPort]::new('COM5',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 14
} finally {
    if ($port.IsOpen) { $port.Close() }
}

```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q450 — Code

```

$port = [IO.Ports.SerialPort]::new('COM6',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 10
} finally {
    if ($port.IsOpen) { $port.Close() }
}

```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q451 — Code

```
$port = [IO.Ports.SerialPort]::new('COM7',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 11
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?**Question EN: Why is the finally block important in this serial test?**

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q452 — Code

```
$port = [IO.Ports.SerialPort]::new('COM8',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 12
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?**Question EN: Why is the finally block important in this serial test?**

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q453 — Code

```
$port = [IO.Ports.SerialPort]::new('COM9',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 13
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?**Question EN: Why is the finally block important in this serial test?**

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q454 — Code

```
$port = [IO.Ports.SerialPort]::new('COM3',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 14
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q455 — Code

```
$port = [IO.Ports.SerialPort]::new('COM4',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 10
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Question EN: Why is the finally block important in this serial test?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q456 — Code

```
$port = [IO.Ports.SerialPort]::new('COM5',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 11
} finally {
```

```
if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q457 — Code

```
$port = [IO.Ports.SerialPort]::new('COM6',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 12
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q458 — Code

```
$port = [IO.Ports.SerialPort]::new('COM7',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 13
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q459 — Code

```
$port = [IO.Ports.SerialPort]::new('COM8',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
```

```
$reply = $port.ReadLine()
if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 14
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

Q460 — Code

```
$port = [IO.Ports.SerialPort]::new('COM9',115200)
try {
    $port.ReadTimeout = 1000
    $port.Open()
    $port.WriteLine('ID?')
    $reply = $port.ReadLine()
    if (-not $reply) { throw 'Empty reply' }
} catch {
    Write-Error "Serial test failed: $($_.Exception.Message)"
    exit 10
} finally {
    if ($port.IsOpen) { $port.Close() }
}
```

Question FR: Pourquoi le bloc finally est-il important dans ce test série ?

Réponse: Il ferme le port qu'une exception survienne ou non, évitant de laisser le COM verrouillé pour le test suivant. Le catch transforme l'erreur en diagnostic explicite et code de sortie, tandis que le timeout empêche un blocage infini.

Question EN: Why is the finally block important in this serial test?

Answer: It closes the port whether or not an exception occurs, preventing the COM port from remaining locked for the next test. The catch turns the failure into explicit diagnostics and an exit code, while the timeout prevents an infinite block.

42. Analyse de code - programmes natifs et exit codes / Code analysis - native programs and exit codes

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q461 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '1')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q462 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify','--port', 'SWD', '--index', '2')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q463 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify','--port', 'SWD', '--index', '3')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q464 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify','--port', 'SWD', '--index', '4')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q465 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify','--port', 'SWD', '--index', '5')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q466 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '6')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q467 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '7')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q468 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '8')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q469 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '9')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q470 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '10')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q471 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '11')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q472 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '12')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q473 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '13')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q474 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '14')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q475 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '15')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q476 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '16')
& $exe @args
$rc = $LASTEXITCODE
```

```
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q477 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '17')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q478 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '18')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q479 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '19')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

Q480 — Code

```
$exe = 'C:\Tools\programmer.exe'
$args = @('--identify', '--port', 'SWD', '--index', '20')
& $exe @args
$rc = $LASTEXITCODE
if ($rc -ne 0) {
    throw "Programmer failed with exit code $rc"
}
```

Question FR: Pourquoi le script capture-t-il \$LASTEXITCODE juste après l'outil natif ?

Question EN: Why does the script capture \$LASTEXITCODE immediately after the native tool?

Réponse: \$LASTEXITCODE appartient au dernier programme natif. Une autre exécution native pourrait le remplacer; le copier immédiatement préserve le résultat exact du programme. Le tableau d'arguments évite de construire une longue commande texte fragile.

Answer: \$LASTEXITCODE belongs to the last native program. Another native invocation could overwrite it, so capturing it immediately preserves the programmer's exact result. The argument array also avoids building a fragile long command string.

43. Analyse de code - registre et configuration Windows / Code analysis - registry and Windows configuration

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q481 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station1'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q482 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station2'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q483 — Code

```
$path = 'HKCU:\Software\LabTool'
```

```
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station3'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q484 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station4'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q485 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station5'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q486 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station6'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q487 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station7'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q488 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station8'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q489 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station9'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q490 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
```

```
Set-ItemProperty -Path $path -Name Mode -Value 'Station10'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q491 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station11'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q492 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station12'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q493 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station13'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q494 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station14'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q495 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station15'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q496 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station16'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q497 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
```



```
Set-ItemProperty -Path $path -Name Mode -Value 'Station17'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q498 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station18'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q499 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station19'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

Q500 — Code

```
$path = 'HKCU:\Software\LabTool'
if (-not (Test-Path $path)) { New-Item $path | Out-Null }
$old = (Get-ItemProperty -Path $path -Name Mode -ErrorAction SilentlyContinue).Mode
Set-ItemProperty -Path $path -Name Mode -Value 'Station20'
$new = (Get-ItemProperty -Path $path -Name Mode).Mode
[pscustomobject]@{Old=$old; New=$new}
```

Question FR: Quels contrôles de traçabilité ce code réalise-t-il et que manque-t-il pour une modification critique ?

Question EN: What traceability checks does this code perform, and what is missing for a critical change?

Réponse: Il lit l'ancienne valeur, écrit la nouvelle et relit pour vérifier. Pour une modification critique, il faudrait aussi journaliser timestamp/utilisateur/version, gérer les erreurs avec try/catch, éventuellement fournir rollback et utiliser -WhatIf/ShouldProcess dans une fonction adaptée.

Answer: It reads the old value, writes the new value, and reads it back for verification. A critical change should also log timestamp/user/version, handle errors with try/catch, possibly support rollback, and use -WhatIf/ShouldProcess in an appropriate function.

44. Analyse de code - réseau et API / Code analysis - networking and APIs

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q501 — Code

```
$target = '192.168.10.21'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST01'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q502 — Code

```
$target = '192.168.10.22'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST02'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q503 — Code

```
$target = '192.168.10.23'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST03'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q504 — Code

```
$target = '192.168.10.24'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST04'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q505 — Code

```
$target = '192.168.10.25'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST05'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q506 — Code

```
$target = '192.168.10.26'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST06'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q507 — Code

```
$target = '192.168.10.27'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST07'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q508 — Code

```
$target = '192.168.10.28'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST08'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q509 — Code

```
$target = '192.168.10.29'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST09'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q510 — Code

```
$target = '192.168.10.30'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST10'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q511 — Code

```
$target = '192.168.10.31'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST11'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q512 — Code

```
$target = '192.168.10.32'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST12'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q513 — Code

```
$target = '192.168.10.33'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST13'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q514 — Code

```
$target = '192.168.10.34'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST14'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q515 — Code

```
$target = '192.168.10.35'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST15'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q516 — Code

```
$target = '192.168.10.36'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST16'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q517 — Code

```
$target = '192.168.10.37'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST17'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q518 — Code

```
$target = '192.168.10.38'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
```

```
}
$payload = [pscustomobject]@{Station='ST18'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q519 — Code

```
$target = '192.168.10.39'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST19'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

Q520 — Code

```
$target = '192.168.10.40'
$tcp = Test-NetConnection -ComputerName $target -Port 502 -WarningAction SilentlyContinue
if (-not $tcp.TcpTestSucceeded) {
    throw "TCP/502 unavailable on $target"
}
$payload = [pscustomobject]@{Station='ST20'; Status='Ready'} | ConvertTo-Json
Invoke-RestMethod -Uri "http://$target/api/status" -Method Post -Body $payload -ContentType 'application/json' | Out-Null
```

Question FR: Pourquoi le script teste-t-il le port TCP avant l'appel REST et quelle limite subsiste ?

Question EN: Why does the script test the TCP port before the REST call, and what limitation remains?

Réponse: Le test TCP sépare rapidement un problème de connectivité/port d'un problème HTTP. Mais un port ouvert ne garantit pas que l'API répond correctement; Invoke-RestMethod doit encore être entouré d'une gestion d'erreur et idéalement de timeouts/retry contrôlés.

Answer: The TCP test quickly separates connectivity/port problems from HTTP problems. But an open port does not guarantee the API works; Invoke-RestMethod still needs error handling and ideally controlled timeouts/retries.

45. Analyse de code - PnP et USB / Code analysis - PnP and USB

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q521 — Code

```
$pattern = 'VID_1201&PID_3401'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q522 — Code

```
$pattern = 'VID_1202&PID_3402'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q523 — Code

```
$pattern = 'VID_1203&PID_3403'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q524 — Code

```
$pattern = 'VID_1204&PID_3404'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q525 — Code

```
$pattern = 'VID_1205&PID_3405'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q526 — Code

```
$pattern = 'VID_1206&PID_3406'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q527 — Code

```
$pattern = 'VID_1207&PID_3407'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q528 — Code

```
$pattern = 'VID_1208&PID_3408'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
}
```



```
Status      = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q529 — Code

```
$pattern = 'VID_1209&PID_3409'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q530 — Code

```
$pattern = 'VID_120A&PID_340A'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q531 — Code

```
$pattern = 'VID_120B&PID_340B'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q532 — Code

```
$pattern = 'VID_120C&PID_340C'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q533 — Code

```
$pattern = 'VID_120D&PID_340D'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q534 — Code

```
$pattern = 'VID_120E&PID_340E'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId    = $dev.InstanceId
    Status        = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q535 — Code

```
$pattern = 'VID_120F&PID_340F'
```

```
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q536 — Code

```
$pattern = 'VID_1210&PID_3410'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q537 — Code

```
$pattern = 'VID_1211&PID_3411'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q538 — Code

```
$pattern = 'VID_1212&PID_3412'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q539 — Code

```
$pattern = 'VID_1213&PID_3413'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

Q540 — Code

```
$pattern = 'VID_1214&PID_3414'
$dev = Get-PnpDevice -PresentOnly | Where-Object InstanceId -match $pattern | Select-Object -First 1
if (-not $dev) { throw 'Fixture USB not found' }
[pscustomobject]@{
    FriendlyName = $dev.FriendlyName
    InstanceId   = $dev.InstanceId
    Status       = $dev.Status
}
```

Question FR: Que vérifie ce code et pourquoi VID/PID seuls peuvent-ils être insuffisants ?

Réponse: Il cherche un périphérique USB/PnP présent dont l'InstanceId contient le VID/PID attendu et retourne son identité/statut. Plusieurs unités identiques partagent VID/PID; pour distinguer un fixture précis il peut falloir numéro de série USB, InstanceId complet ou propriété de localisation.

Question EN: What does this code verify, and why can VID/PID alone be insufficient?

Answer: It finds a present USB/PnP device whose InstanceId contains the expected VID/PID and returns identity/status. Multiple identical units share VID/PID, so distinguishing one fixture may require USB serial number, full InstanceId, or location information.

46. Analyse de code - protocoles série / Code analysis - serial protocols

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q541 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM5', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
```

}

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q542 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM6', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q543 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM7', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q544 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM8', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
```

```

    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}

```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q545 — Code

```

$sp = [IO.Ports.SerialPort]::new('COM9', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}

```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q546 — Code

```

$sp = [IO.Ports.SerialPort]::new('COM4', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}

```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q547 — Code

```

$sp = [IO.Ports.SerialPort]::new('COM5', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()

```

```
$sp.WriteLine('MEAS?')
$reply = $sp.ReadLine().Trim()
$value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q548 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM6', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q549 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM7', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q550 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM8', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
```

```
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et logger la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q551 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM9', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et logger la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q552 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM4', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et logger la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q553 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM5', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q554 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM6', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q555 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM7', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q556 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM8', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q557 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM9', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Question EN: Explain the precautions taken before accepting the serial measurement.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q558 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM4', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q559 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM5', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

Q560 — Code

```
$sp = [IO.Ports.SerialPort]::new('COM6', 115200, 'None', 8, 'One')
$sp.ReadTimeout = 1500
try {
    $sp.Open()
    $sp.DiscardInBuffer()
    $sp.WriteLine('MEAS?')
    $reply = $sp.ReadLine().Trim()
    $value = [double]::Parse($reply, [Globalization.CultureInfo]::InvariantCulture)
    if ($value -lt 2.8 -or $value -gt 3.4) { throw "Out of range: $value" }
} finally {
    if ($sp.IsOpen) { $sp.Close() }
}
```

Question FR: Expliquez les précautions prises avant d'accepter la mesure série.

Réponse: Le script fixe timeout, vide l'ancien buffer, envoie une requête, nettoie la réponse, parse avec culture invariante puis applique des limites. Le finally libère le port. Dans une vraie station, il faudrait aussi gérer explicitement timeout/format invalides et loguer la valeur brute.

Question EN: Explain the precautions taken before accepting the serial measurement.

Answer: The script sets a timeout, discards stale input, sends a request, trims the reply, parses using invariant culture, and applies limits. Finally releases the port. A real station should also explicitly handle timeout/invalid format and log the raw value.

47. Analyse de code - fonctions de test / Code analysis - test functions

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q561 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
```

```
[Parameter(Mandatory)][double]$Value,
[double]$Min = 3.1,
[double]$Max = 3.5
)
[pscustomobject]@{
    Name='VDD1'
    Value=$Value
    Min=$Min
    Max=$Max
    Pass=($Value -ge $Min -and $Value -le $Max)
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q562 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD2'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q563 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD3'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q564 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD4'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q565 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD5'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q566 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD6'
        Value=$Value
    }
}
```

```

    Min=$Min
    Max=$Max
    Pass=($Value -ge $Min -and $Value -le $Max)
}
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q567 — Code

```

function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD7'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q568 — Code

```

function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD8'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q569 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD9'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q570 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD10'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q571 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD11'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q572 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD12'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q573 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD13'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q574 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
}
```



```

)
[pscustomobject]@{
    Name='VDD14'
    Value=$Value
    Min=$Min
    Max=$Max
    Pass=($Value -ge $Min -and $Value -le $Max)
}
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q575 — Code

```

function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD15'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q576 — Code

```

function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD16'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}

```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q577 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD17'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q578 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD18'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q579 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD19'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

}

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

Q580 — Code

```
function Test-Voltage {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)][double]$Value,
        [double]$Min = 3.1,
        [double]$Max = 3.5
    )
    [pscustomobject]@{
        Name='VDD20'
        Value=$Value
        Min=$Min
        Max=$Max
        Pass=($Value -ge $Min -and $Value -le $Max)
    }
}
```

Question FR: Pourquoi cette fonction retourne-t-elle un objet plutôt qu'un simple PASS/FAIL ?

Réponse: L'objet conserve valeur mesurée, limites, nom et décision. Il peut être exporté, affiché, envoyé au serveur ou agrégé avec d'autres tests sans perdre l'information nécessaire au diagnostic.

Question EN: Why does this function return an object instead of a simple PASS/FAIL?

Answer: The object preserves the measured value, limits, test name, and decision. It can be exported, displayed, sent to a server, or combined with other tests without losing diagnostic information.

48. Analyse de code - jobs et parallélisme / Code analysis - jobs and parallelism

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q581 — Code

```
$targets = 'PC-A1','PC-B1','PC-C1'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q582 — Code

```
$targets = 'PC-A2','PC-B2','PC-C2'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
}
```

```

    }
} -ThrottleLimit 3
$results

```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q583 — Code

```

$targets = 'PC-A3','PC-B3','PC-C3'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results

```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q584 — Code

```

$targets = 'PC-A4','PC-B4','PC-C4'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results

```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q585 — Code

```

$targets = 'PC-A5','PC-B5','PC-C5'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results

```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q586 — Code

```
$targets = 'PC-A6','PC-B6','PC-C6'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q587 — Code

```
$targets = 'PC-A7','PC-B7','PC-C7'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q588 — Code

```
$targets = 'PC-A8','PC-B8','PC-C8'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q589 — Code

```
$targets = 'PC-A9','PC-B9','PC-C9'
```

```
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q590 — Code

```
$targets = 'PC-A10','PC-B10','PC-C10'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q591 — Code

```
$targets = 'PC-A11','PC-B11','PC-C11'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q592 — Code

```
$targets = 'PC-A12','PC-B12','PC-C12'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q593 — Code

```
$targets = 'PC-A13','PC-B13','PC-C13'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q594 — Code

```
$targets = 'PC-A14','PC-B14','PC-C14'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q595 — Code

```
$targets = 'PC-A15','PC-B15','PC-C15'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q596 — Code

```
$targets = 'PC-A16','PC-B16','PC-C16'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q597 — Code

```
$targets = 'PC-A17','PC-B17','PC-C17'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q598 — Code

```
$targets = 'PC-A18','PC-B18','PC-C18'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q599 — Code

```
$targets = 'PC-A19','PC-B19','PC-C19'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
}
```



```
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

Q600 — Code

```
$targets = 'PC-A20','PC-B20','PC-C20'
$results = $targets | ForEach-Object -Parallel {
    [pscustomobject]@{
        Computer = $_
        Reachable = Test-Connection $_ -Count 1 -Quiet
    }
} -ThrottleLimit 3
$results
```

Question FR: Pourquoi ce parallélisme est-il acceptable ici mais potentiellement dangereux pour un instrument partagé ?

Réponse: Chaque itération teste une cible réseau indépendante, donc le parallélisme réduit le temps. Si toutes les itérations commandaient le même port COM/instrument/fixture, elles pourraient entrelacer les commandes et corrompre le protocole; il faudrait sérialiser l'accès.

Question EN: Why is this parallelism acceptable here but potentially dangerous for a shared instrument?

Answer: Each iteration checks an independent network target, so parallelism reduces time. If all iterations controlled the same COM port/instrument/fixture, commands could interleave and corrupt the protocol; access should be serialized.

49. Analyse de code - programmation firmware / Code analysis - firmware programming

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q601 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_01.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q602 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_02.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q603 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_03.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q604 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_04.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q605 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_05.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q606 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_06.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q607 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_07.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q608 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_08.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q609 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_09.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q610 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_10.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q611 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_11.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q612 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_12.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q613 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_13.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
```

```
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q614 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_14.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q615 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_15.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q616 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_16.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Question EN: What integrity and traceability guarantees does this workflow provide?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q617 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_17.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q618 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_18.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q619 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_19.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

Q620 — Code

```
$fw = Join-Path $PSScriptRoot 'firmware\board_20.hex'
if (-not (Test-Path $fw)) { throw 'Firmware file missing' }
$hash = (Get-FileHash $fw -Algorithm SHA256).Hash
& 'C:\Tools\flash.exe' --erase --write $fw --verify
$rc = $LASTEXITCODE
if ($rc -ne 0) { throw "Flash failed RC=$rc" }
[pscustomobject]@{Firmware=(Split-Path $fw -Leaf); SHA256=$hash; Result='PASS'}
```

Question FR: Quelles garanties de traçabilité et d'intégrité ce workflow apporte-t-il ?

Réponse: Il vérifie l'existence de l'image, calcule son SHA-256, lance erase/write/verify, capture l'exit code et retourne nom/hash/résultat. Pour être complet, il faut aussi enregistrer tool version, device ID, firmware version lue après boot et station/serial number.

Question EN: What integrity and traceability guarantees does this workflow provide?

Answer: It checks that the image exists, calculates SHA-256, runs erase/write/verify, captures the exit code, and returns name/hash/result. A complete workflow should also record tool version, device ID, firmware version read after boot, and station/serial number.

50. Analyse de code - production et traçabilité / Code analysis - production and traceability

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q621 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST01'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.1.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q622 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST02'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.2.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q623 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST03'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.3.0'
```



```

    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")

```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q624 — Code

```

param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST04'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.4.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")

```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q625 — Code

```

param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST05'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.5.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")

```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q626 — Code

```

param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST06'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.6.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")

```


Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q627 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST07'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.7.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q628 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST08'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.8.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q629 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST09'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.9.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q630 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST10'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.10.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q631 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST11'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.11.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q632 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST12'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.12.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q633 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST13'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.13.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q634 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST14'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.14.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q635 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST15'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.15.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q636 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST16'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.16.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q637 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST17'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.17.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q638 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST18'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.18.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q639 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST19'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.19.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

Q640 — Code

```
param([Parameter(Mandatory)][string]$Serial)
if ($Serial -notmatch '^SN[0-9]{8}$') { throw 'Invalid serial format' }
$result = [pscustomobject]@{
    Serial = $Serial
    Station = 'ST20'
    Time = (Get-Date).ToString('o')
    ScriptVersion = '1.20.0'
    Pass = $true
}
$result | ConvertTo-Json | Set-Content (Join-Path $PSScriptRoot "logs\$Serial.json")
```

Question FR: Pourquoi cette structure est-elle adaptée à la traçabilité et quelles informations pourraient encore manquer ?

Question EN: Why is this structure suitable for traceability, and what information could still be missing?

Réponse: Elle valide le SN et sauvegarde station, timestamp, version de script et résultat dans un format structuré. On peut encore ajouter firmware hash/version, fixture ID, operator, mesures brutes, limites/version de configuration et historique de retest.

Answer: It validates the serial number and stores station, timestamp, script version, and result in structured form. It could also include firmware hash/version, fixture ID, operator, raw measurements, limits/config version, and retest history.

51. Débogage - trouvez et corrigez / Debugging - find and fix

Le code est volontairement affiché une seule fois. / The code is intentionally shown only once.

Q641 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.

Question EN: Find the logic error.

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q642 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q643 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q644 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q645 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q646 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q647 — Code

```
$x = 10
if ($x = 5) { 'Five' }
```

Question FR: Trouvez l'erreur logique.**Question EN: Find the logic error.**

Réponse: L'opérateur = affecte 5 à \$x au lieu de comparer. La comparaison correcte est \$x -eq 5. Avec Set-StrictMode et des tests, ce type d'erreur est plus facile à détecter.

Answer: The = operator assigns 5 to \$x instead of comparing. The correct comparison is \$x -eq 5. Set-StrictMode and tests make this kind of error easier to catch.

Q648 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q649 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q650 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q651 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q652 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q653 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q654 — Code

```
Get-Process | Format-Table Name,Id | Export-Csv processes.csv -NoTypeInfo
```

Question FR: Pourquoi le CSV obtenu n'est-il pas celui attendu ?

Question EN: Why is the resulting CSV not what was expected?

Réponse: Format-Table transforme les objets métier en objets de formatage destinés à l'écran. Il faut exporter avant le formatage: Get-Process | Select-Object Name,Id | Export-Csv ...

Answer: Format-Table turns business objects into display-formatting objects. Export before formatting: Get-Process | Select-Object Name,Id | Export-Csv ...

Q655 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Q656 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Q657 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Q658 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Q659 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Q660 — Code

```
# Script opened in PowerShell ISE
$data = 1..20 | ForEach-Object -Parallel { $_ * 2 }
```

Question FR: Pourquoi ce script peut-il échouer dans PowerShell ISE alors qu'il fonctionne sous PowerShell 7 ?

Question EN: Why can this script fail in PowerShell ISE while working in PowerShell 7?

Réponse: ForEach-Object -Parallel est une fonctionnalité de PowerShell moderne. PowerShell ISE exécute Windows PowerShell 5.1, qui ne la prend pas en charge. Il faut exécuter le script sous pwsh/VS Code ou écrire une alternative compatible 5.1.

Answer: ForEach-Object -Parallel is a modern PowerShell feature. PowerShell ISE runs Windows PowerShell 5.1, which does not support it. Run the script under pwsh/VS Code or write a 5.1-compatible alternative.

Quick Interview Review / Révision rapide

- Core: objects, pipeline, variables, types, functions, modules, error handling, files, JSON/CSV, regex.
- Windows: processes, services, registry, network, WMI/CIM, PnP/USB, Event Logs, scheduled tasks.
- Hardware: SerialPort, programmer CLIs, instrument automation, safe state, timeouts, firmware validation.
- Production: structured results, serial number, station ID, tool/firmware/script versions, logs, PASS/FAIL rules.
- Environment: Windows PowerShell 5.1 vs PowerShell 7; PowerShell ISE vs VS Code; compatibility must be tested.